МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «ДОНБАССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет [факультет — уточнить]
Кафедра [кафедра — уточнить]

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

на тему «Интегрирование функций, заданных формулой»

по дисциплине «Основы программирования»

Разработал: студент гр. БИ-25 Кириенко А.М.

Руководитель: [должность — уточнить]
Самойлов Д.В.

Алчевск 2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине «Основы программирования»

Студент: Кириенко Артём Максимович, группа БИ-25
Вариант 7. Тема: «Интегрирование функций, заданных формулой».

Разработать программу на языке высокого уровня (C#.NET), которая выполняет:

1. ввод данных из файла *.txt;
2. выбор метода интегрирования из двух;
3. вывод результатов на форму и в файл *.html.

Интерфейс: главная форма и подчинённые формы (MDI), главное меню, панель инструментов, строка состояния, справка о работе с программой.

Календарный план

№ п/п	Наименование этапов курсового проекта (работы)	Срок выполнения этапов	Примечание
1	Описание задачи, разработка содержательной блок-схемы алгоритма программы	1 неделя	
2	Определение и описание переменных, констант и подпрограмм (функций)	3 неделя	КТ
3	Разработка программы интерфейса	5 неделя	
4	Разработка программного кода основного алгоритма	7 неделя	КТ
5	Разработка программного кода обработчиков событий	9, 10 неделя	КТ
6	Выполнение тестового примера	11, 12 неделя	КТ
7	Оформление записки объяснительной	13 неделя	
8	Сдача работы на проверку	14 неделя	КТ
9	Защита курсовой работы	15, 16 недели	КТ

Студент _____ «____» _____ 2026 г.

Руководитель _____ Самойлов Д.В. «____» _____ 2026 г.

РЕФЕРАТ

Пояснительная записка – 48 с., количество источников – 3, таблиц – 15, рисунков – 20.

Объект разработки – прикладное программное обеспечение, разработанное с использованием языка программирования высокого уровня C# (платформа .NET Framework 4.8, библиотека Windows Forms).

Цель работы – разработка программы для вычисления определённых интегралов функций, заданных формулой, методами трапеций и Симпсона.

Основными характеристиками программы «Интегрирование функций» являются: ввод формулы подынтегральной функции в виде строки с контролем синтаксиса и области определения; ввод исходных данных из текстового файла; выбор метода – трапеций, Симпсона или оба метода для сравнения; расчёт при заданном числе разбиений или с автоматическим подбором числа разбиений под заданную точность на основе правила Рунге; предупреждение при негладкой функции, для которой оценка погрешности занижена; вывод результатов в таблицы окна, на график и в файл формата HTML. Интерфейс программы многооконный (MDI) с главным меню, панелью инструментов, строкой состояния и справкой. Вычислительное ядро программы проверено автоматическими модульными тестами.

ЧИСЛЕННОЕ ИНТЕГРИРОВАНИЕ, МЕТОД ТРАПЕЦИЙ, МЕТОД СИМПСОНА, ПРАВИЛО РУНГЕ, ЛЕКСИЧЕСКИЙ АНАЛИЗ, СИНТАКСИЧЕСКИЙ АНАЛИЗ, MDI-ИНТЕРФЕЙС

СОДЕРЖАНИЕ

Задание и календарный план....2
Реферат....3
Введение....5
1 Анализ задания....7
1.1 Теоретические сведения, необходимые для выполнения задания....7
1.2 Концепция решения задания....11
2 Разработка программы....14
2.1 Общие требования к программе и её структура....14
2.2 Разработка интерфейса пользователя....16
2.3 Описание переменных и констант....18
2.4 Алгоритмы расчётов....22
3 Реализация и тестирование программы....31
3.1 Описание подпрограмм....31
3.2 Управляющая программа и обработчики событий....33
3.3 Тестирование программы....34
Выводы....37
Список источников....38
Приложение А. Экранные формы....39
Приложение Б. Листинг программы....40
Приложение В. Файлы тестового примера....47

ВВЕДЕНИЕ

Определённый интеграл встречается в большинстве инженерных и научных расчётов: с его помощью находят площади и объёмы, работу силы, вероятность попадания случайной величины в заданный интервал, действующее значение переменного сигнала. Формула Ньютона–Лейбница позволяет вычислить интеграл точно, но только тогда, когда известна первообразная подынтегральной функции в элементарных функциях. Для многих практически важных функций, например e^(−x²) или sin x / x, такой первообразной не существует. Если функция задана таблицей или алгоритмом, о первообразной вообще нельзя говорить. В этих случаях интеграл вычисляют приближённо [1, 2].

Численное интегрирование заменяет интеграл конечной суммой значений подынтегральной функции в узлах разбиения отрезка. Различные способы выбора узлов и весов дают квадратурные формулы с разной точностью и разной трудоёмкостью. Две наиболее распространённые формулы – формула трапеций, в которой функция на каждом частичном отрезке заменяется отрезком прямой, и формула Симпсона, в которой она заменяется параболой. При одинаковом числе узлов формула Симпсона обычно даёт заметно меньшую погрешность, но для функций с особенностями это преимущество может исчезнуть. Поэтому программа, которая позволяет сравнить оба метода на одной и той же функции, наглядно показывает связь между гладкостью функции, порядком метода и достижимой точностью.

Практическое применение численных методов требует двух дополнительных возможностей. Во-первых, пользователь должен задавать функцию в привычной математической записи, а не в виде текста программы. Для этого программа должна разобрать введённую строку, проверить её синтаксис и вычислить значение выражения при любом x, то есть решить задачу лексического и синтаксического анализа [3]. Во-вторых, результат должен сопровождаться оценкой погрешности: число без сведений о его точности малоценно.

Задание на курсовую работу (вариант 7) предусматривает разработку программы интегрирования функций, заданных формулой: ввод исходных данных из текстового файла, выбор одного из двух методов интегрирования, вывод результатов на форму и в файл формата HTML. Программа должна иметь многооконный интерфейс с главным меню, панелью инструментов, строкой состояния и справкой о работе с программой.

Цель работы – разработать такую программу и показать на ней работу методов трапеций и Симпсона. Для достижения цели необходимо решить следующие задачи:

– изучить квадратурные формулы трапеций и Симпсона, оценки их погрешности и правило Рунге;

– разработать алгоритмы лексического и синтаксического анализа формулы и вычисления её значения;

– реализовать вычислительное ядро программы с контролем области определения функции и ошибок ввода;

– разработать многооконный интерфейс, вывод результатов на форму, на график и в файл HTML;

– подготовить тестовые примеры, в том числе с негладкими функциями и ошибочными данными, и проверить правильность работы программы.

Программа разработана на языке C# для платформы .NET Framework 4.8 с использованием библиотеки Windows Forms. Вычислительная часть выделена в отдельную библиотеку, не зависящую от интерфейса, и проверяется автоматическими модульными тестами. В первом разделе записки изложены теоретические сведения и концепция решения, во втором – требования к программе, интерфейс пользователя, описание переменных и констант и алгоритмы, в третьем – программная реализация и тестирование.

1 Анализ задания

1.1 Теоретические сведения, необходимые для выполнения задания

1.1.1 Задача численного интегрирования

Требуется вычислить определённый интеграл

$$I = \int f(x)\,dx,\ \ x \in [a;\ b] \qquad (1.1)$$

от функции f(x), непрерывной на отрезке [a; b]. Если a > b, интеграл по определению меняет знак, а при a = b он равен нулю. Отрезок делят на n равных частей с шагом

$$h = (b - a) / n,\ \ x_i = a + i \cdot h,\ \ i = 0, 1, \ldots, n \qquad (1.2)$$

и записывают интеграл как сумму интегралов по частичным отрезкам $[x_{i-1};\ x_i]$. На каждом из них функцию заменяют многочленом невысокой степени, интеграл от которого вычисляется точно. Получающаяся сумма

$$Q = c_0 \cdot f(x_0) + c_1 \cdot f(x_1) + \ldots + c_n \cdot f(x_n) \qquad (1.3)$$

называется квадратурной формулой, числа c_i – её весами [1]. Формулу считают имеющей порядок точности p, если погрешность составной формулы убывает как h^p при уменьшении шага. Чем выше порядок, тем быстрее приближённое значение сходится к точному при росте n, но тем более гладкой должна быть функция.

1.1.2 Формула трапеций

На частичном отрезке $[x_{i-1};\ x_i]$ функцию заменяют линейной, проходящей через точки $(x_{i-1};\ f_{i-1})$ и $(x_i;\ f_i)$, где $f_i = f(x_i)$. Площадь получившейся трапеции равна $h \cdot (f_{i-1} + f_i) / 2$. После суммирования по всем частичным отрезкам получается составная формула трапеций:

$$T_n = h \cdot [\ (f_0 + f_n) / 2 + f_1 + f_2 + \ldots + f_{n-1}\ ] \qquad (1.4)$$

Для дважды непрерывно дифференцируемой функции погрешность равна [1]

$$I - T_n = -(b - a) \cdot h^2 \cdot f''(\xi) / 12,\ \ \xi \in [a;\ b] \qquad (1.5)$$

Метод имеет второй порядок точности: при удвоении n погрешность уменьшается примерно в 4 раза. Формула точна для линейных функций, так как для них f″ = 0. Для вычисления T_n требуется n + 1 значение функции.

1.1.3 Формула Симпсона

На сдвоенном отрезке $[x_{i-1};\ x_{i+1}]$ длиной 2h функцию заменяют параболой, проходящей через три точки. Интеграл от этой параболы равен $h \cdot (f_{i-1} + 4f_i + f_{i+1}) / 3$. Для разбиения на сдвоенные отрезки число n должно быть чётным. Суммируя по всем сдвоенным отрезкам, получают составную формулу Симпсона:

$$S_n = h/3 \cdot [\ f_0 + 4 \cdot (f_1 + f_3 + \ldots + f_{n-1}) + 2 \cdot (f_2 + f_4 + \ldots + f_{n-2}) + f_n\ ] \qquad (1.6)$$

Для четырежды непрерывно дифференцируемой функции погрешность равна [1]

$$I - S_n = -(b - a)\cdot h^4 \cdot f^{(4)}(\xi) / 180,\ \ \xi \in [a; b] \qquad (1.7)$$

Метод имеет четвёртый порядок точности: при удвоении n погрешность уменьшается примерно в 16 раз. Формула точна не только для парабол, но и для кубических многочленов, поскольку четвёртая производная многочлена третьей степени равна нулю. Между формулами трапеций и Симпсона существует связь

$$S_{2n} = (4\cdot T_{2n} - T_n) / 3 \qquad (1.8)$$

то есть формула Симпсона получается из двух формул трапеций исключением главного члена погрешности.

Пример. Вычислим интеграл ∫ e^(−x²) dx на отрезке [0; 2] при n = 4, h = 0,5. Точное значение интеграла равно (√π / 2)·erf(2) = 0,8820813908. Значения функции в узлах приведены в таблице 1.1.

Таблица 1.1 – Значения подынтегральной функции в узлах при n = 4

i	x_i	$f_i = e^{(-x_i^2)}$	Вес в формуле Симпсона
0	0	1,000000	1
1	0,5	0,778801	4
2	1	0,367879	2
3	1,5	0,105399	4
4	2	0,018316	1

По формулам (1.4) и (1.6) получаем:

$$T_4 = 0{,}5\cdot[\ (1 + 0{,}018316) / 2 + 0{,}778801 + 0{,}367879 + 0{,}105399\ ] = 0{,}880619 \qquad (1.9)$$

$$S_4 = 0{,}5/3\cdot[\ 1 + 4\cdot(0{,}778801 + 0{,}105399) + 2\cdot 0{,}367879 + 0{,}018316\ ] = 0{,}881812 \qquad (1.10)$$

Погрешность формулы трапеций $1{,}46\cdot10^{-3}$, формулы Симпсона $2{,}69\cdot10^{-4}$: при одном и том же числе узлов формула Симпсона точнее примерно в пять раз. В таблице 1.2 показано, как погрешности меняются при удвоении n. Отношение погрешностей для формулы трапеций стремится к 4, для формулы Симпсона – к 16, что подтверждает второй и четвёртый порядок точности. Значения в таблице получены разработанной программой.

Таблица 1.2 – Сходимость методов для интеграла e^(−x²) на отрезке [0; 2]

n	T_n	$I - T_n$	Отн.	S_n	$I - S_n$	Отн.
4	0,880618634	$1{,}46\cdot10^{-3}$	—	0,881812425	$2{,}69\cdot10^{-4}$	—
8	0,881703791	$3{,}78\cdot10^{-4}$	3,87	0,882065510	$1{,}59\cdot10^{-5}$	16,94
16	0,881986245	$9{,}51\cdot10^{-5}$	3,97	0,882080397	$9{,}94\cdot10^{-7}$	15,97
32	0,882057558	$2{,}38\cdot10^{-5}$	3,99	0,882081329	$6{,}21\cdot10^{-8}$	16,01
64	0,882075430	$5{,}96\cdot10^{-6}$	4,00	0,882081387	$3{,}88\cdot10^{-9}$	16,00

Преимущество метода Симпсона заметно и при заданной точности: для этого интеграла при ε = 10^{-8} метод трапеций потребовал n = 2048 разбиений, а метод Симпсона – n = 64.

1.1.4 Оценка погрешности. Правило Рунге

Выражения (1.5) и (1.7) содержат производные функции в неизвестной точке ξ, поэтому на практике ими пользоваться трудно. Применяют правило Рунге. Пусть погрешность формулы порядка p равна $I - Q_n \approx C \cdot h^p$, причём коэффициент C слабо зависит от h. Тогда для вдвое меньшего шага $I - Q_{2n} \approx C \cdot (h/2)^p$, и разность двух приближений $Q_{2n} - Q_n = (I - Q_n) - (I - Q_{2n}) \approx C \cdot h^p \cdot (1 - 2^{-p})$. Отсюда [2]

$$I - Q_{2n} \approx (Q_{2n} - Q_n) / (2^p - 1) \qquad (1.11)$$

Для формулы трапеций p = 2 и знаменатель равен 3, для формулы Симпсона p = 4 и знаменатель равен 15. Оценка (1.11) относится к уточнённому значению Q_{2n}. Погрешность значения Q_n в 2^p раз больше: $I - Q_n \approx 2^p \cdot (Q_{2n} - Q_n) / (2^p - 1)$.

Правило Рунге позволяет подбирать число разбиений автоматически. Вычисление начинают с некоторого n, затем n удваивают и сравнивают два последних приближения. Процесс прекращается, когда оценка (1.11) не превосходит заданной точности ε либо число разбиений достигает предельного значения.

Правило Рунге верно только тогда, когда погрешность действительно убывает как h^p, то есть для достаточно гладких функций. Если у функции в какой-либо точке отрезка бесконечная производная, например y √x при x = 0, фактический порядок точности ниже теоретического, а оценка (1.11) оказывается заниженной. Фактический порядок можно определить по трём последовательным приближениям:

$$p^* = \log_2 \left[(Q_{n/2} - Q_n) / (Q_n - Q_{2n}) \right] \qquad (1.12)$$

Для гладкой функции p^* близко к p, для негладкой оно заметно меньше. Пример: интеграл функции $f(x) = 20 \cdot \sin(3\sqrt{x})$ на отрезке [0; 10], взятой из задания на курсовую работу. Подстановка $x = t^2$ даёт точное значение $I = 40 \cdot (\sin 3t / 9 - t \cdot \cos 3t / 3)$ при $t = \sqrt{10}$, то есть I = 41,8069225426. В таблице 1.3 приведены результаты, полученные при $\varepsilon = 10^{-6}$; для обоих методов фактический порядок равен 1,5.

Таблица 1.3 – Оценка погрешности и истинная погрешность для функции $20 \cdot \sin(3\sqrt{x})$

Метод	Значение интеграла	Оценка по Рунге	Истинная погрешность
Трапеций (n = 409 600)	41,8069210375	$9{,}18 \cdot 10^{-7}$	$1{,}51 \cdot 10^{-6}$
Симпсона (n = 102 400)	41,8069178417	$5{,}73 \cdot 10^{-7}$	$4{,}70 \cdot 10^{-6}$

Истинная погрешность в 1,6 и 8,2 раза превышает оценку, поэтому для негладких функций программа должна не только выводить оценку по правилу Рунге, но и предупреждать пользователя о сниженном порядке сходимости.

1.1.5 Анализ выражений, заданных формулой

Пользователь вводит функцию как строку символов, например 20*sin(sqrt(x)*3). Чтобы вычислить её значение, программа решает три задачи [3]. Лексический анализ разбивает строку на лексемы – неделимые элементы

выражения: числа, имена функций и констант, знаки операций, скобки. Синтаксический анализ проверяет, что последовательность лексем соответствует грамматике языка выражений, и строит структуру выражения. Вычисление определяет числовое значение по этой структуре.

Грамматику записывают в нормальной форме Бэкуса–Наура (БНФ). Нетерминальные символы – понятия языка – заключают в угловые скобки: <Выражение>. Терминальные символы, то есть символы самого выражения, заключают в одинарные кавычки: '+'. Знак ::= означает «определяется как», вертикальная черта – «или», квадратные скобки обозначают необязательную часть, фигурные – повторение ноль или более раз.

Для анализа по такой грамматике удобен метод рекурсивного спуска: каждому нетерминальному символу ставится в соответствие функция, которая разбирает соответствующую часть выражения и при необходимости вызывает функции других символов. Приоритет операций задаётся вложенностью правил: чем ниже в грамматике находится операция, тем выше её приоритет. Повторение в правилах реализуется циклом, что устраняет левую рекурсию и сохраняет левую ассоциативность сложения, вычитания, умножения и деления.

Результатом разбора служит дерево выражения. Его узлами являются число, переменная, унарный минус, бинарная операция и вызов функции. Значение выражения находится обходом дерева от листьев к корню. Строку нужно разбирать один раз: при вычислении интеграла значения функции требуются в тысячах и миллионах точек, и повторный разбор строки в каждой из них замедлил бы расчёт во столько раз, сколько операций содержит выражение.

Для выражения 20*sin(sqrt(x)*3) дерево показано на рисунке 1.1: корень – умножение, его операнды – число 20 и вызов функции sin, аргумент которой – произведение вызова sqrt и числа 3.

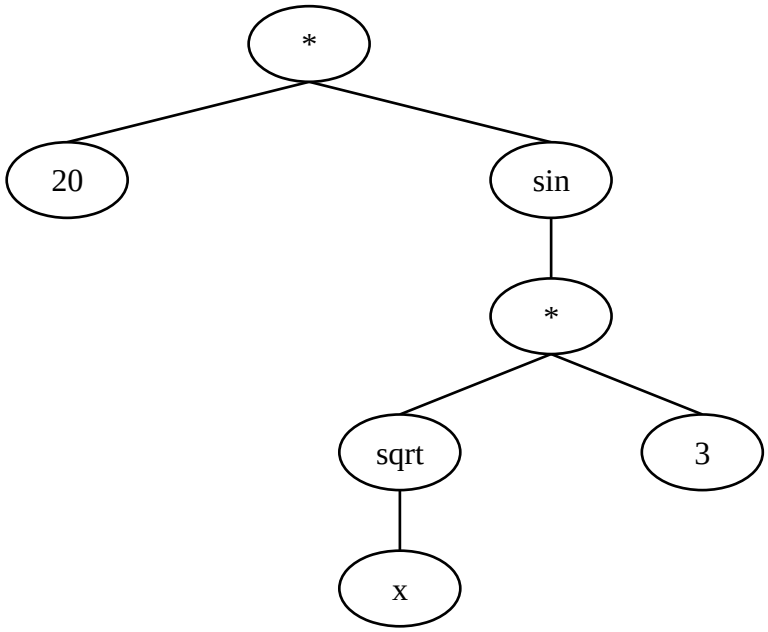


Рисунок 1.1 – Дерево выражения 20*sin(sqrt(x)*3)

При вычислении необходимо контролировать область определения: деление на ноль, логарифм неположительного числа, корень из отрицательного числа, арксинус и арккосинус вне отрезка [−1; 1], тангенс и котангенс в точках разрыва, степень отрицательного числа с нецелым показателем. При нарушении любого из условий вычисление прекращается с сообщением, содержащим значение x.

1.2 Концепция решения задания

Программа разрабатывается на языке C# для платформы .NET Framework 4.8 с использованием библиотеки Windows Forms. Язык разрешён заданием на курсовую работу, библиотека содержит готовые компоненты многооконного интерфейса: форму-контейнер дочерних окон, главное меню, панель инструментов и строку состояния. Платформа входит в состав Windows 10 и 11, поэтому программа запускается без установки среды разработки.

Программа состоит из двух частей. Вычислительное ядро оформлено как отдельная библиотека. В неё входят лексический и синтаксический анализаторы формулы, вычисление дерева выражения, методы трапеций и Симпсона, чтение и запись файла исходных данных, подготовка данных графика и формирование отчёта в формате HTML. Интерфейс пользователя образует приложение с главным окном, окнами расчёта и графика, справкой и окном «О программе». Такое разделение даёт три преимущества: ядро проверяется автоматическими модульными тестами без запуска окон; график в окне и график в отчёте строятся по одним и тем же данным; ядро можно использовать в другом приложении.

Формула функции записывается по грамматике, приведённой на рисунке 1.2. Регистр букв в именах функций и констант не учитывается. Десятичной точкой служит точка или запятая. Знак умножения нужно писать явно, запись 2x недопустима. Приоритеты операций показаны в таблице 1.4, допустимые функции и константы – в таблице 1.5.

```
<Выражение> ::= <Слагаемое> { ( '+' | '-' ) <Слагаемое> }
<Слагаемое> ::= <Множитель> { ( '*' | '/' ) <Множитель> }
<Множитель> ::= [ '+' | '-' ] <Степень>
<Степень>   ::= <Основа> [ '^' <Множитель> ]
<Основа>    ::= <Число> | 'x' | <Константа>
              | <Функция> '(' <Выражение> ')' | '(' <Выражение> ')'
<Функция>   ::= 'sin' | 'cos' | 'tg' | 'ctg' | 'arcsin' | 'arccos'
              | 'arctg' | 'exp' | 'ln' | 'lg' | 'sqrt' | 'abs'
<Константа> ::= 'pi' | 'e'
<Число>     ::= <Цифры> [ ( '.' | ',' ) <Цифры> ]
              [ ( 'e' | 'E' ) [ '+' | '-' ] <Цифры> ]
<Цифры>     ::= <Цифра> { <Цифра> }
<Цифра>     ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

Рисунок 1.2 – Грамматика формулы в БНФ

Таблица 1.4 – Приоритет и ассоциативность операций

Приоритет	Операция	Порядок выполнения
1 (высший)	скобки, вызов функции	изнутри наружу
2	возведение в степень ^	справа налево: 2^3^2 = 2^9
3	унарные + и −	применяются к степени: −x^2 = −(x^2)
4	умножение * и деление /	слева направо
5 (низший)	сложение + и вычитание −	слева направо

Таблица 1.5 – Функции и константы формулы

Запись	Назначение	Область определения
sin(x), cos(x)	синус, косинус (радианы)	любое x
tg(x), ctg(x)	тангенс, котангенс	вне точек разрыва
arcsin(x), arccos(x)	арксинус, арккосинус	$-1 \le x \le 1$
arctg(x)	арктангенс	любое x
exp(x)	экспонента e^x	любое x (результат конечный)
ln(x), lg(x)	натуральный и десятичный логарифм	$x > 0$
sqrt(x)	квадратный корень	$x \ge 0$
abs(x)	модуль	любое x
pi, e	числа π и e	—

Исходные данные читаются из текстового файла, состоящего из строк «ключ = значение»; символ # начинает комментарий. Обязательны формула и пределы интегрирования, остальные параметры – метод, число разбиений n и точность ε – необязательны. Пример файла приведён на рисунке 1.3. При ошибке в файле программа указывает номер строки, при ошибке в формуле – позицию символа.

```
# Исходные данные: интегрирование функции, заданной формулой
# Функция из примечания к заданию курсовой работы
function = 20*sin(sqrt(x)*3)
a        = 0
b        = 10
method   = both        # trapezoid | simpson | both
n        = 100         # число разбиений (начальное, если задана eps)
eps      = 1e-6        # необязательно: требуемая точность
```

Рисунок 1.3 – Пример файла исходных данных

Расчёт выполняется в одном из двух режимов. Если точность ε не задана, интеграл вычисляется при заданном числе разбиений n, а оценка погрешности находится по правилу Рунге для выводимого значения. Если ε задана, число разбиений удваивается до достижения точности; все промежуточные приближения выводятся в таблице сходимости. Предельное число разбиений ограничено значением $2^{20} = 1\,048\,576$, чтобы расчёт не мог продолжаться неограниченно долго. Пользователь может выбрать метод трапеций, метод Симпсона или оба метода сразу для сравнения. Для метода Симпсона нечётное n автоматически увеличивается до чётного с сообщением пользователю. Если фактический порядок сходимости заметно ниже теоретического, программа выводит предупреждение о негладкой функции.

Результаты выводятся тремя способами: в таблицы окна расчёта, на график с закрашенной площадью под кривой и в файл формата HTML. Отчёт HTML представляет собой один файл со встроенными стилями и графиком в формате SVG, поэтому его можно открыть на любом компьютере и распечатать из браузера. Работоспособность программы проверяется модульными тестами на контрольных интегралах с известным значением, на порядок точности методов и на все виды ошибок ввода.

2 Разработка программы

2.1 Общие требования к программе и её структура

Требования к программе взяты из задания на курсовую работу (вариант 7), методических указаний по курсовому проектированию и из решений, принятых при анализе задания в разделе 1. Функциональные и эксплуатационные требования сведены в таблицу 2.1.

Таблица 2.1 – Требования к программе

№	Требование	Как выполнено
1	Функция задаётся формулой в виде строки	Собственный лексический и синтаксический анализатор: классы Lexer и Parser, дерево выражения
2	Ввод исходных данных из файла *.txt	Класс TaskFile: строки «ключ = значение», сообщения с номером строки
3	Выбор метода из двух: трапеций и Симпсона	Список «Метод» в окне расчёта и меню «Расчёт – Метод»; режим «Оба метода» для сравнения
4	Вывод результатов на форму	Таблицы результатов и сходимости, предупреждения, график в отдельном окне
5	Вывод результатов в файл *.html	Класс HtmlReport: один файл со встроенными стилями и графиком SVG
6	Контроль ввода и понятные сообщения об ошибках	Исключения с кодом ошибки и позицией; все тексты собраны в классе Messages
7	Многооконный (MDI) интерфейс, главное меню, панель инструментов, строка состояния	Класс MainForm: каждый расчёт открывается в собственном дочернем окне
8	Справка о работе с программой	Окно справки с деревом разделов, вызов по F1 и из окон ввода
9	Горячие клавиши основных команд	Ctrl+N, Ctrl+O, Ctrl+S, Ctrl+E, F5, F6, F1
10	Таблица и график не размещаются на одной форме	График выводится в отдельном дочернем окне GraphForm
11	Размер модуля не более 250 строк, венгерская нотация имён элементов форм, комментарии	Проверяется скриптом tools/check_lines.sh; элементы форм: txtFormula, cmbMethod, btnCalculate и т. д.

Программа состоит из двух проектов, связанных так, как показано на рисунке 2.1. Проект Integral.Core – библиотека классов без единой формы: она содержит всю вычислительную часть, формирование графика и HTML-отчёта и не знает о существовании окон. Проект Integral.App – приложение Windows Forms, которое только обращается к ядру и показывает его результаты. Такое разделение позволило проверить ядро автоматическими модульными тестами без запуска интерфейса и использовать одну и ту же модель графика в окне и в отчёте.

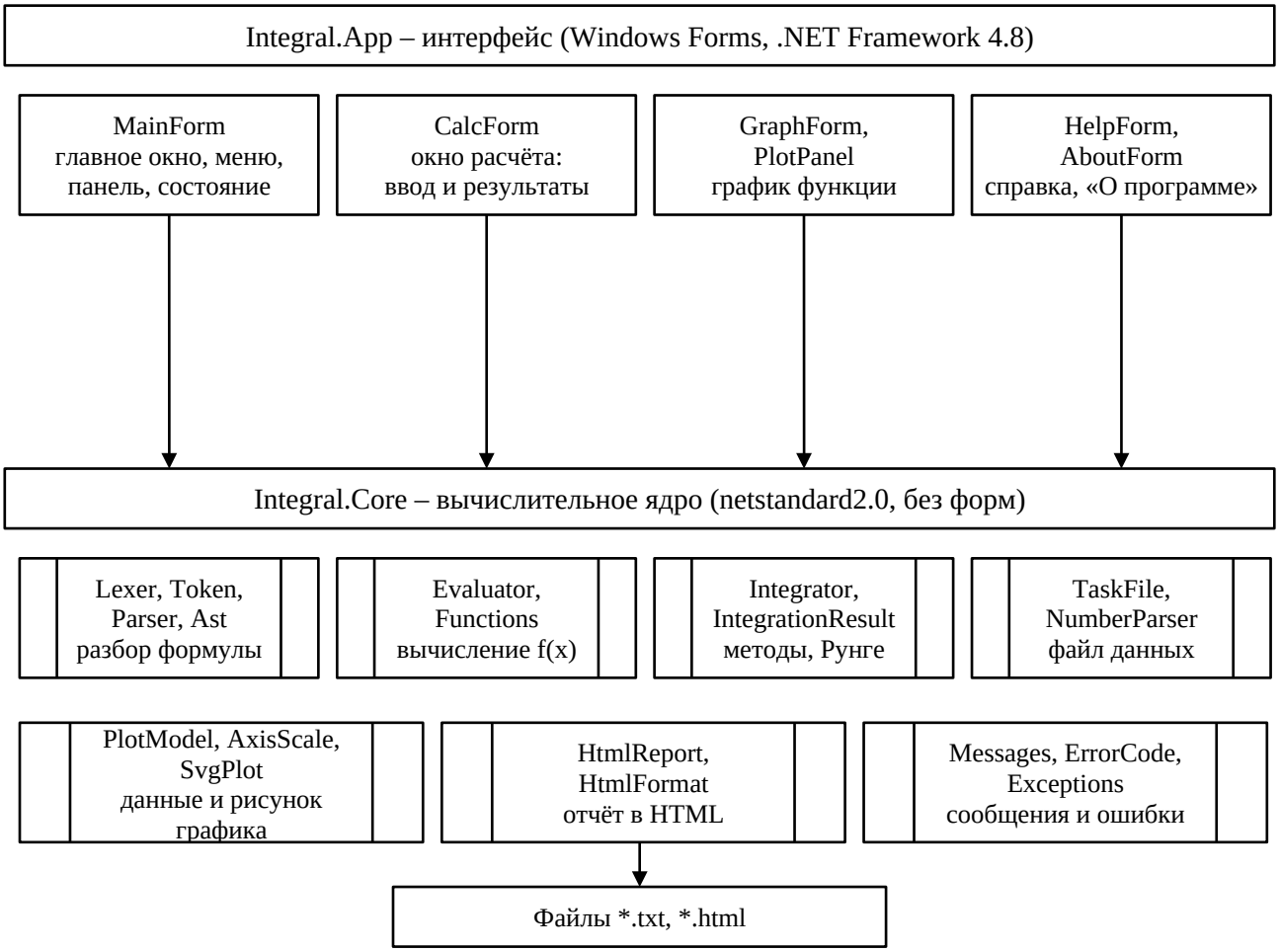


Рисунок 2.1 – Структура программы

Назначение основных модулей приведено в таблице 2.2. Каждый модуль занимает не более 250 строк; крупные классы разделены на части (например, MainForm, MainForm.Designer и MainForm.Files).

Таблица 2.2 – Основные модули программы

Модуль	Назначение
Lexer, Token	Разбиение формулы на лексемы: числа, имена, знаки операций, скобки
Parser, Ast	Построение дерева выражения методом рекурсивного спуска по грамматике БНФ
Evaluator, Functions	Вычисление дерева в заданной точке с проверкой области определения
Integrator, IntegrationResult	Формулы трапеций и Симпсона, правило Рунге, таблица сходимости
TaskFile, NumberParser	Чтение и запись файла исходных данных; разбор чисел из полей ввода
PlotModel, AxisScale, SvgPlot	Данные графика (точки, диапазоны, деления осей) и его запись в формате SVG
HtmlReport, HtmlFormat	Сборка самодостаточного HTML-отчёта; форматирование чисел
Messages, ErrorCode, Exceptions	Коды ошибок, их русские тексты, классы исключений
MainForm	Главное окно: меню, панель инструментов, строка состояния, управление дочерними окнами
CalcForm	Окно расчёта: поля ввода, запуск расчёта, таблицы результатов
GraphForm, PlotPanel	Окно графика и рисование модели графика средствами GDI+
HelpForm, AboutForm, Ui	Справка, сведения о программе, общие размеры и шрифты форм

2.2 Разработка интерфейса пользователя

Интерфейс построен по принципу MDI: главное окно содержит меню, панель инструментов и строку состояния, а каждый расчёт открывается в отдельном дочернем окне. Можно держать открытыми несколько расчётов с разными функциями и сравнивать их. Формы выдержаны в единой манере: светло-серый фон, белые поля ввода, шрифт Tahoma размером 10 пунктов, подписи выровнены по левому краю, связанные элементы собраны в группы с заголовками. Высота поля ввода составляет 20 пикселей, интервал между связанными элементами – 5, между несвязанными – 8 пикселей.

2.2.1 Главное окно

Главное меню состоит из четырёх пунктов: «Файл», «Расчёт», «Окно» и «Справка». Команды меню и клавиши быстрого доступа приведены в таблице 2.3. Во всех пунктах меню работают подчёркнутые буквы (клавиша Alt).

Таблица 2.3 – Команды главного меню

Меню	Команда	Клавиши	Действие
Файл	Новый расчёт	Ctrl+N	открыть новое окно расчёта
Файл	Открыть данные…	Ctrl+O	загрузить файл *.txt в новое окно
Файл	Сохранить данные…	Ctrl+S	записать исходные данные в *.txt
Файл	Экспорт отчёта в HTML…	Ctrl+E	сохранить результаты в *.html
Файл	Выход	Alt+F4	закрыть программу
Расчёт	Вычислить	F5	выполнить расчёт
Расчёт	Метод – трапеций / Симпсона / оба	–	выбрать метод
Расчёт	Показать график	F6	открыть окно графика
Окно	Каскадом, мозаикой по горизонтали и по вертикали, упорядочить значки, закрыть все	–	расположение дочерних окон; внизу список открытых окон
Справка	Содержание	F1	справка о работе с программой
Справка	Формат входного файла	–	справка на разделе о файле данных
Справка	О программе	–	сведения о программе и авторе

Панель инструментов дублирует основные команды значками: «Новый расчёт», «Открыть данные», «Сохранить данные», «Экспорт в HTML», «Вычислить», «Показать график», «Справка», «О программе». У каждого значка есть всплывающая подсказка с названием команды и клавишами. Команды, недоступные в текущем состоянии (например, экспорт при отсутствии открытых окон), отключаются в меню и на панели одновременно.

Строка состояния разделена на три части: слева выводится подсказка к выбранной команде или результат последнего действия, в середине – сообщение о состоянии расчёта («Готово: …, I = …», «Данные изменены», «Ошибка в формуле», «Отчёт сохранён»), справа – выбранный метод. Об ошибках ввода программа сообщает диалоговым окном, после чего переводит курсор в неверное поле; при ошибке в формуле курсор ставится на неверный символ.

[Здесь будет снимок: главное окно с открытым окном расчёта – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 2.2 – Главное окно программы

2.2.2 Окно расчёта

Окно расчёта содержит три группы (рисунок 2.3). В группе «Функция и пределы» находятся поле формулы f(x), поля пределов a и b и серая подсказка о допустимых функциях и знаках. В группе «Метод и точность» – список методов («Трапеций», «Симпсона», «Оба метода»), поля числа разбиений n и точности ε, кнопка «Вычислить» и пояснение: если ε не задана, расчёт идёт при указанном n, иначе n подбирается автоматически. В группе «Результат» – сводная таблица (значение интеграла, число разбиений, шаг, оценка погрешности, число

вычислений функции, время), строка предупреждений и таблица сходимости выбранного метода.

Числа можно вводить с точкой или запятой. Поля проверяются при нажатии кнопки «Вычислить»: при ошибке программа показывает понятное сообщение, а в сообщении об ошибке формулы называет позицию неверного символа. Если после расчёта изменить любое поле, результаты помечаются как устаревшие; экспорт и график в этом случае предлагают пересчитать данные. Окно расчёта можно изменять по размеру: таблицы растягиваются вместе с окном.

[Здесь будет снимок: окно расчёта с результатами для функции из задания – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 2.3 – Окно расчёта

2.2.3 Окно графика, справка и сведения о программе

График (рисунок 2.4) выводится в отдельном дочернем окне, чтобы таблицы и график не мешали друг другу. На нём показаны кривая функции, закрашенная площадь под кривой, оси с подписями и, если число разбиений невелико (не более 64), узлы разбиения. Масштаб подбирается автоматически; если функция имеет полюс и уходит в бесконечность, диапазон по оси y определяется по основной массе значений, чтобы один выброс не превратил график в линию. При изменении размера окна график перерисовывается.

[Здесь будет снимок: окно графика функции – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 2.4 – Окно графика

Справка открывается по F1 в отдельном окне: слева дерево разделов (начало работы, формат файла, синтаксис формул, методы и точность, сообщения об ошибках, горячие клавиши), справа текст раздела. Из окна расчёта справка открывается на разделе, относящемся к текущему полю. Окно «О программе» – диалог со сведениями о программе, авторе и руководителе.

Файл данных можно открыть тремя способами: командой «Открыть данные», сочетанием Ctrl+O и перетаскиванием файла *.txt из Проводника в главное окно. Результат можно сохранить в файл *.html командой «Экспорт отчёта», после чего отчёт открывается в браузере.

2.3 Описание переменных и констант

Основные переменные программы приведены в таблице 2.4, константы и перечисления – в таблице 2.5. Имя дано в виде «Класс.Член». Локальные переменные малого размера (счётчики циклов, промежуточные суммы формул) описаны только для основного алгоритма Integrator.Integrate.

Таблица 2.4 – Основные переменные

Имя	Диапазон значений	Тип данных	Описание
`Lexer._src`	строка	string	Анализируемая формула
`Lexer._pos`	0…длина формулы	int	Индекс следующего символа формулы
`Token.Type`	Number…End (12 значений)	TokenType	Тип лексемы
`Token.Position`	1…длина+1	int	Позиция лексемы в формуле, считая с 1
`Token.Length`	0…длина формулы	int	Длина лексемы в символах
`Token.Value`	любое double	double	Значение числа или константы (pi, e)
`Token.Name`	строка	string	Имя функции или константы
`Parser._lexer`	объект	Lexer	Источник лексем
`Parser._cur`	лексема	Token	Текущая лексема разбора
`Parser._depth`	0…200	int	Текущая вложенность разбора
`IntegrationParams.A`	конечное число	double	Нижний предел интегрирования
`IntegrationParams.B`	конечное число	double	Верхний предел интегрирования
`IntegrationParams.N`	1…1 048 576	int	Число разбиений (начальное, если задана точность)
`IntegrationParams.Eps`	≥ 0, конечное	double	Требуемая точность; 0 — расчёт с фиксированным n
`IntegrationResult.Value`	любое double	double	Значение интеграла
`IntegrationResult.N`	1…1 048 576	int	Итоговое число разбиений
`IntegrationResult.H`	любое double	double	Итоговый шаг (B − A) / N
`IntegrationResult.ErrorEstimate`	≥ 0 или NaN	double	Оценка погрешности по правилу Рунге
`IntegrationResult.ObservedOrder`	double или NaN	double	Фактический порядок сходимости
`IntegrationResult.OrderDegraded`	False…True	bool	Фактический порядок ниже теоретического (оценка занижена)
`IntegrationResult.Evaluations`	$0…2^{63}-1$	long	Число вычислений подынтегральной функции
`IntegrationResult.ElapsedMs`	≥ 0	double	Время расчёта, мс
`IntegrationResult.Converged`	False…True	bool	Достигнута ли заданная точность
`IntegrationResult.NAdjusted`	False…True	bool	n увеличено до чётного (метод Симпсона)
`IntegrationResult.Steps`	список	IReadOnlyList<ConvergenceStep>	Таблица сходимости
`ConvergenceStep.ErrorEstimate`	≥ 0 или NaN	double	Оценка погрешности на шаге (NaN на первом)
`IntegrationTask.Formula`	строка	string	Формула подынтегральной функции
`IntegrationTask.Method`	Trapezoid, Simpson, Both	MethodChoice	Выбранный метод

Имя	Диапазон значений	Тип данных	Описание
`EvalException.X`	любое double	double	Значение x, при котором возникла ошибка
`FormulaException.Position`	1…длина+1	int	Позиция ошибки в формуле
`TaskFileException.Line`	0…число строк	int	Номер строки файла с ошибкой (0 — файл в целом)
`Integrator.evals`	$0…2^{63}-1$	long	Счётчик вычислений f(x) в Integrate
`Integrator.order`	2 или 4	int	Порядок точности выбранного метода
`Integrator.divisor`	3 или 15	double	Знаменатель $2^p - 1$ в правиле Рунге
`Integrator.prev`	любое double	double	Значение интеграла на предыдущем шаге
`Integrator.cur`	любое double	double	Значение интеграла при удвоенном n
`Integrator.n2`	2…1 048 576	int	Удвоенное число разбиений
`Integrator.steps`	список	List<ConvergenceStep>	Накопленная таблица сходимости
`Integrator.h`	любое double	double	Шаг разбиения в формулах трапеций и Симпсона
`Integrator.sum`	любое double	double	Накопленная сумма значений функции
`TaskFile.seen`	словарь	Dictionary<string,int>	Ключи, уже встреченные в файле, и номера строк
`TaskFile.lineNo`	1…число строк	int	Номер разбираемой строки файла

Таблица 2.5 – Константы и перечисления

Имя	Значение	Тип данных	Описание
`IntegrationParams.MaxN`	1 << 20 = 1 048 576	int	Предел удвоения числа разбиений
`Parser.MaxDepth`	200	int	Предельная вложенность разбора (защита от переполнения стека)
`Functions.PoleEps`	10^{-12}	double	Порог близости к полюсу для tg и ctg
`Functions.Table`	12 функций	Dictionary<string, Func<double,double>>	sin, cos, tg, ctg, arcsin, arccos, arctg, exp, ln, lg, sqrt, abs
`Functions.Constants`	pi, e	Dictionary<string,double>	Именованные константы формулы
`TaskFile.Keys`	6 ключей	string[]	function, a, b, method, n, eps
`Ui.ControlHeight`	20	int	Высота поля ввода, пикс. (методические указания, табл. 2.1)
`Ui.GapRelated`	5	int	Интервал между связанными элементами, пикс.
`Ui.GapUnrelated`	8	int	Интервал между несвязанными элементами, пикс.

Имя	Значение	Тип данных	Описание
`PlotModel.MaxNodes`	64	int	Наибольшее n, при котором на графике показываются узлы разбиения
`TokenType`	12 значений	перечисление	Number, Variable, Constant, Function, Plus, Minus, Multiply, Divide, Power, LeftParen, RightParen, End
`Method`	2 значения	перечисление	Trapezoid (порядок 2), Simpson (порядок 4)
`MethodChoice`	3 значения	перечисление	Trapezoid, Simpson, Both
`ErrorCode`	27 значений	перечисление	Коды ошибок разбора, вычисления, параметров и файла (тексты — в Messages)

2.4 Алгоритмы расчётов

Общая схема работы программы показана на рисунке 2.5. Пользователь вводит формулу, пределы, метод и параметры точности; программа проверяет данные и разбирает формулу в дерево выражения. Затем выбранным методом (или обоими) вычисляется интеграл, при ошибке в вычислении функции выводится сообщение, иначе результаты выводятся на форму и по желанию сохраняются в файл HTML.

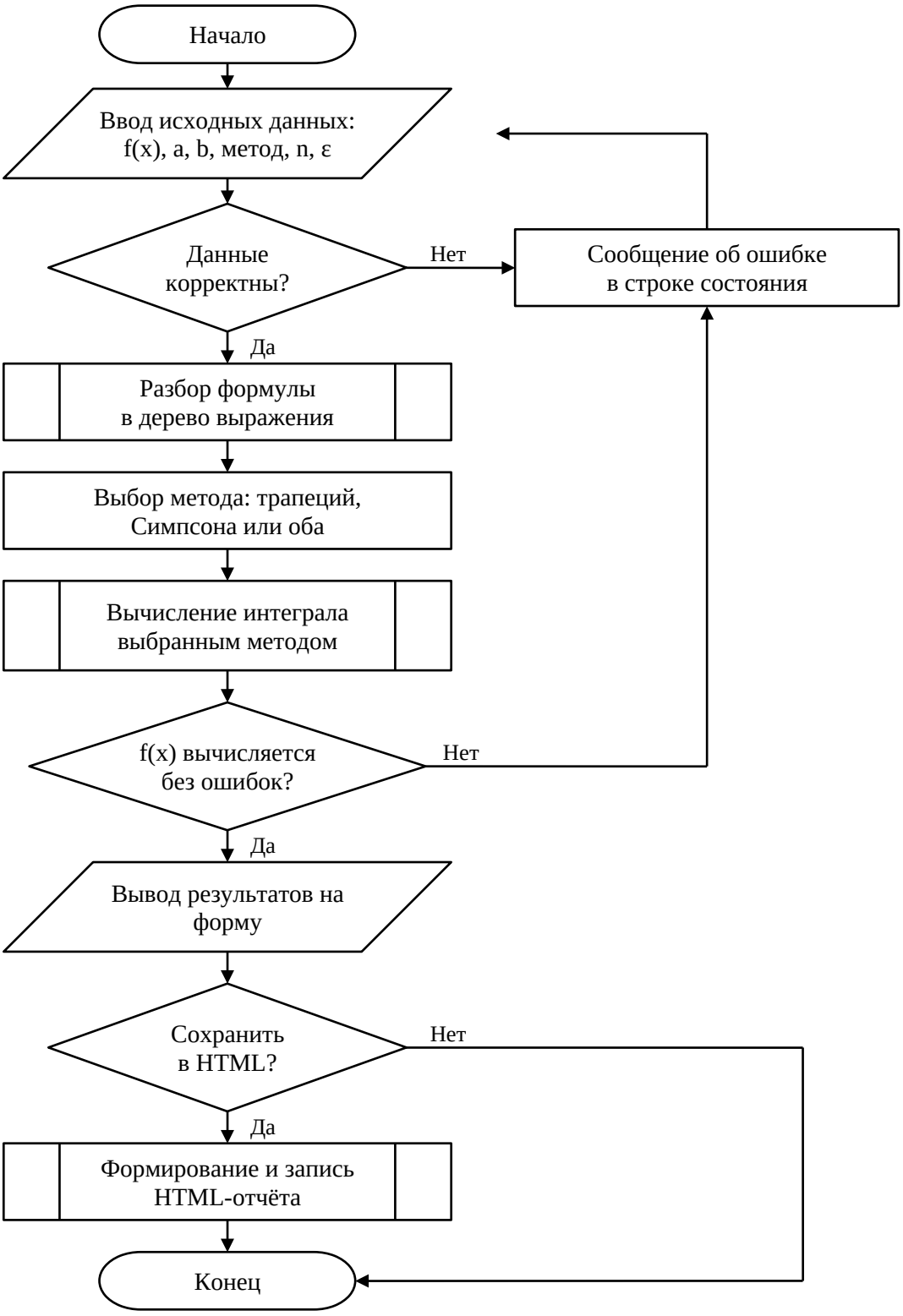


Рисунок 2.5 – Общая схема работы программы

2.4.1 Лексический анализ

Метод Lexer.Next (рисунок 2.6) выделяет из строки следующую лексему. Сначала пропускаются пробелы. Если строка закончилась, возвращается лексема конца. Знаки операций и скобки возвращаются сразу. Цифра начинает число: метод ReadNumber читает целую часть, дробную часть после точки или запятой и порядок после e или E, причём порядок принимается, только если за ним идут цифры (иначе e – отдельная константа, как в записи 2*e). Буква начинает слово: метод ReadWord определяет, что это – переменная x, константа pi или e, имя одной из двенадцати функций или неизвестное имя, которое считается ошибкой. Любой другой символ приводит к ошибке «недопустимый символ» с его позицией. Длина лексем и их позиции запоминаются для сообщений об ошибках.

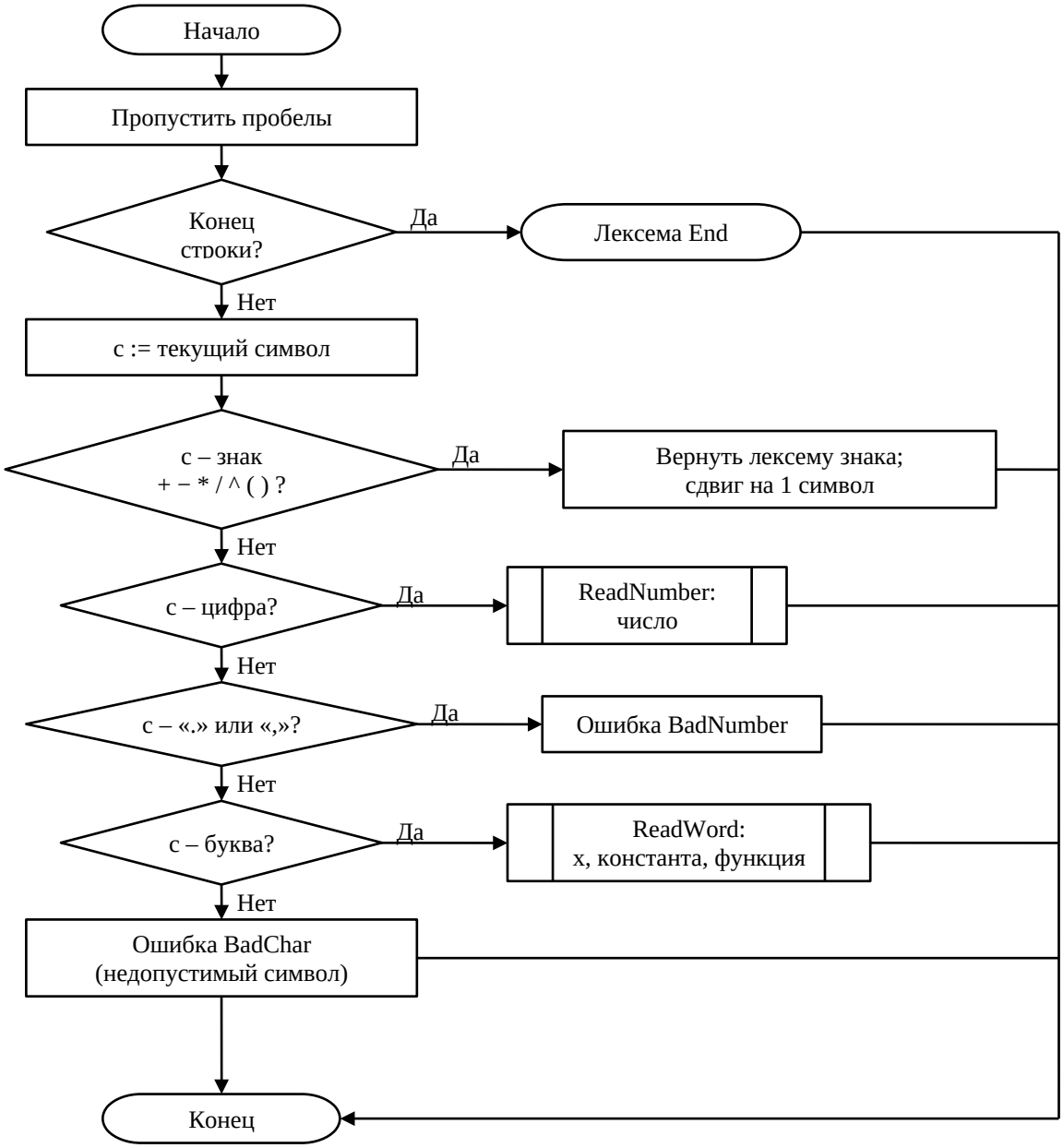


Рисунок 2.6 – Алгоритм выделения лексемы Lexer.Next

2.4.2 Синтаксический анализ

Разбор выполняется методом рекурсивного спуска [3]: каждому правилу грамматики из рисунка 1.2 соответствует метод класса Parser, который вызывает методы правил низшего приоритета (ParseExpression – ParseTerm – ParseFactor – ParsePower – ParseBase). Результат разбора – дерево выражения из узлов NumberNode, VariableNode, NegateNode, BinaryNode и FunctionNode.

Алгоритм метода ParseExpression показан на рисунке 2.7. Метод разбирает первое слагаемое, а затем, пока текущая лексема – плюс или минус, запоминает знак, разбирает следующее слагаемое и объединяет оба в узел бинарной операции, так что операции одного приоритета выполняются слева направо. Счётчик вложенности увеличивается при входе и уменьшается при выходе; если он превышает 200, выдаётся ошибка «формула слишком сложна», что защищает программу от переполнения стека при записи вида «((((…».

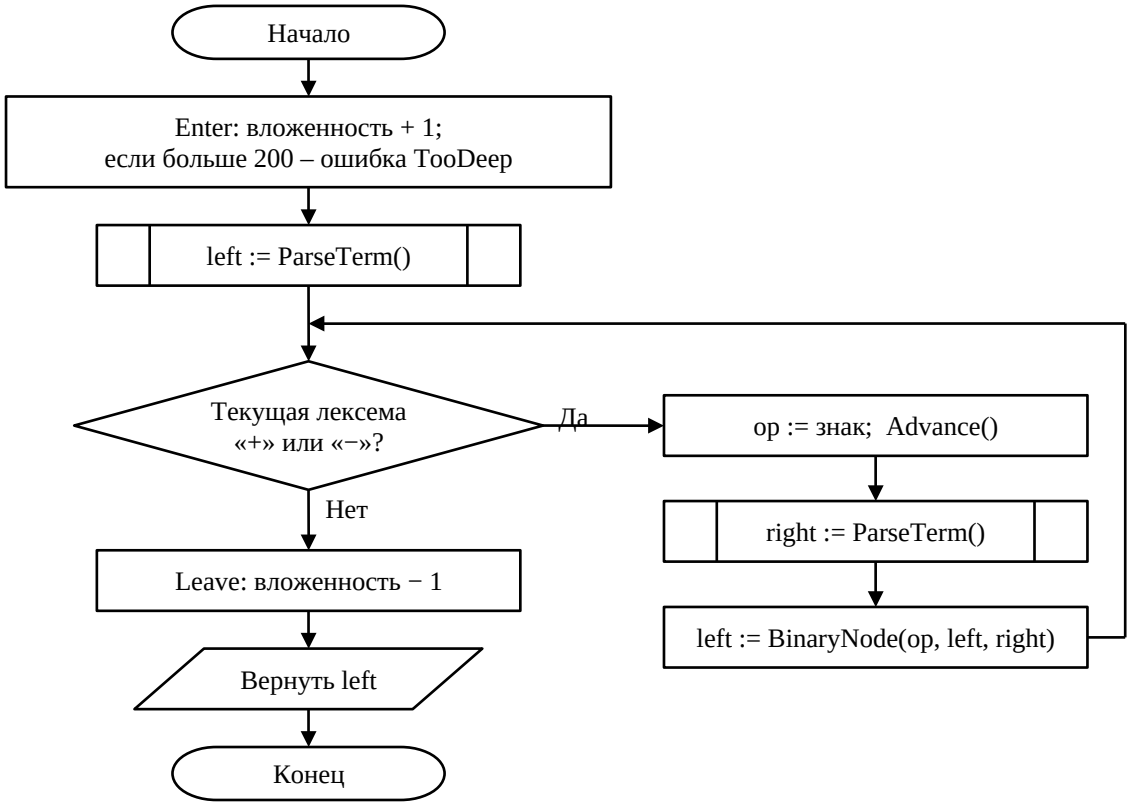


Рисунок 2.7 – Алгоритм метода ParseExpression

Метод ParseTerm построен так же, но обрабатывает знаки умножения и деления и вызывает ParseFactor. Метод ParseFactor обрабатывает необязательный знак перед степенью: минус создаёт узел NegateNode, плюс игнорируется. Поэтому запись −x^2 разбирается как −(x²). Метод ParsePower разбирает основу и, если за ней следует знак «^», показатель, причём показатель разбирается методом ParseFactor, что даёт правую ассоциативность: 2^3^2 = 2^9.

Метод ParseBase (рисунок 2.8) разбирает основу выражения. Число или константа превращаются в узел NumberNode, переменная x – в VariableNode. Имя функции должно сопровождаться открывающей скобкой; внутри скобок вызывается ParseExpression и проверяется закрывающая скобка. Выражение в скобках разбирается так же. Любая другая лексема вызывает ошибку «ожидается число, переменная x, функция или «(»». После разбора Parser.Parse проверяет, что разобрана вся строка; лишние символы дают ошибку «недопустимая лексема».

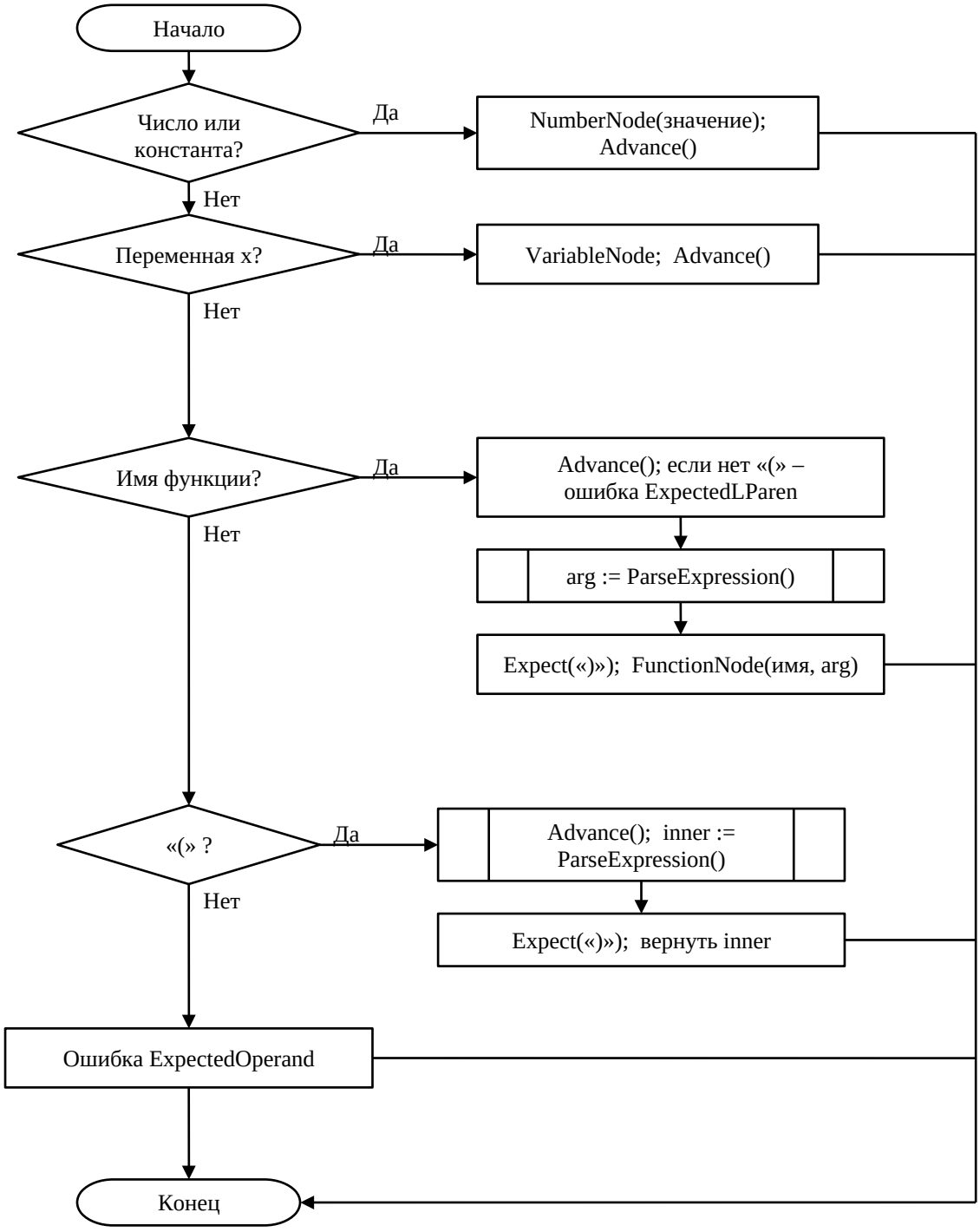


Рисунок 2.8 – Алгоритм метода ParseBase

2.4.3 Вычисление значения выражения

Значение функции в точке x находит метод Evaluator.Evaluate (рисунок 2.9) обходом дерева: для числа возвращается его значение, для переменной – x, для унарного минуса – значение операнда с обратным знаком. Для бинарных операций и функций вызываются вспомогательные методы Binary и Function, которые сначала вычисляют операнды, а затем проверяют условия, приведённые в таблице 2.6. При нарушении любого условия создаётся исключение EvalException с кодом ошибки и значением x, при котором она возникла, так что пользователь видит, в какой точке отрезка функция не определена.

Таблица 2.6 – Проверки области определения

Операция или функция	**Условие ошибки**	**Код ошибки**
`a / b`	b = 0	DivByZero
`a ^ b`	a = 0 и b < 0	DivByZero
`a ^ b`	a < 0 и b не целое	PowDomain
`ln(x), lg(x)`	x ≤ 0	LnDomain
`sqrt(x)`	x < 0	SqrtDomain
`arcsin(x), arccos(x)`	|x| > 1	AsinDomain
`tg(x)`	|cos x| < 10^{-12}	TanPole
`ctg(x)`	|sin x| < 10^{-12}	TanPole
`любая операция`	результат NaN или бесконечность	NotFinite

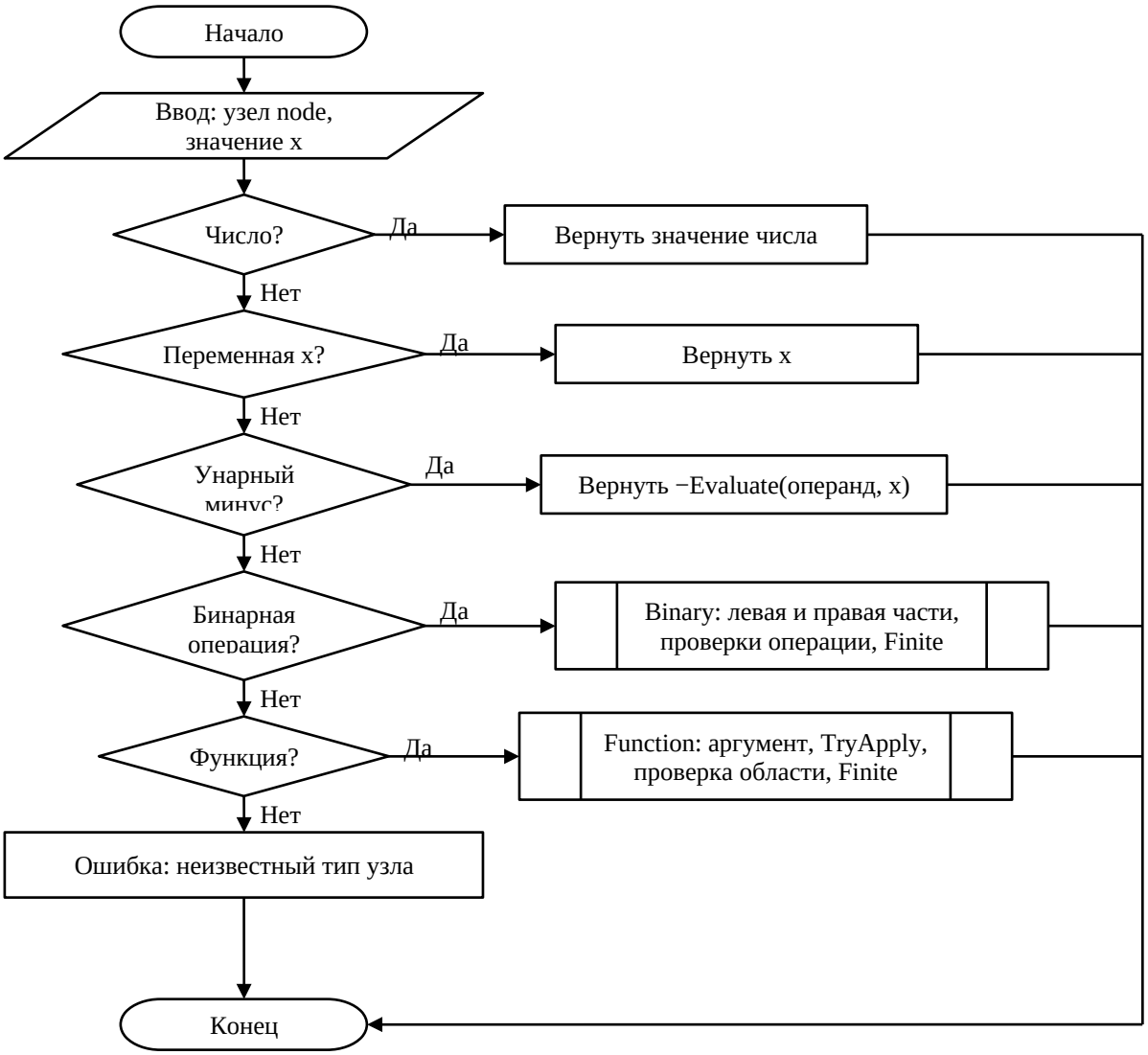


Рисунок 2.9 – Алгоритм метода Evaluator.Evaluate

2.4.4 Вычисление интеграла

Основной алгоритм – метод Integrator.Integrate (рисунок 2.10). Он получает дерево выражения, пределы, число разбиений, точность и метод. Сначала проверяются параметры: пределы должны быть конечными числами, n – целым от 1 до 2^{20}, точность – неотрицательной. Для метода Симпсона n должно быть чётным, поэтому нечётное n увеличивается на единицу, о чём пользователь получает сообщение. Если a = b, интеграл равен нулю.

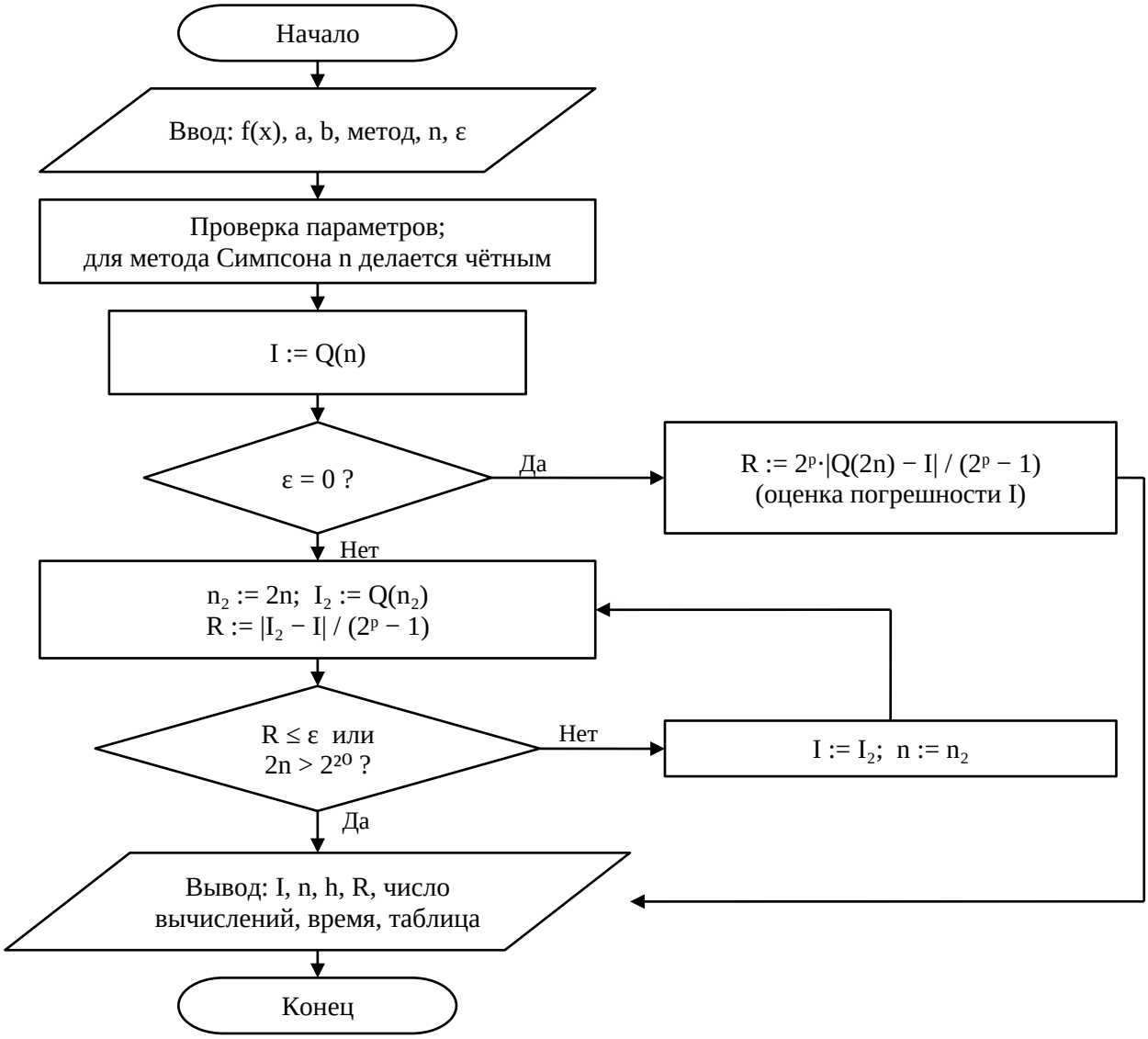


Рисунок 2.10 – Алгоритм метода Integrator.Integrate

Если точность не задана, значение I вычисляется при заданном n, а оценка погрешности находится по правилу Рунге для выводимого значения: $R = 2^p \cdot |Q(2n) - Q(n)| / (2^p - 1)$, где p – порядок метода. Множитель 2^p нужен потому, что разность $|Q(2n) - Q(n)| / (2^p - 1)$ оценивает ошибку более точного значения Q(2n), а ошибка значения Q(n) в 2^p раз больше. Если точность задана, число разбиений удваивается, пока оценка $|Q(2n) - Q(n)| / (2^p - 1)$ не станет не больше ε или пока удвоение не выйдет за предел 2^{20}; последний шаг даёт результат, а все шаги записываются в таблицу сходимости. По трём последним шагам определяется фактический

порядок сходимости; если он меньше 0,8 теоретического, программа предупреждает о негладкой функции.

Составные формулы реализованы методами Integrator.Trapezoid (рисунок 2.11) и Integrator.Simpson (рисунок 2.12). Оба вычисляют шаг h, начальную сумму и в цикле по внутренним узлам добавляют значения функции; в формуле Симпсона узлы с нечётными номерами входят с весом 4, с чётными – с весом 2.

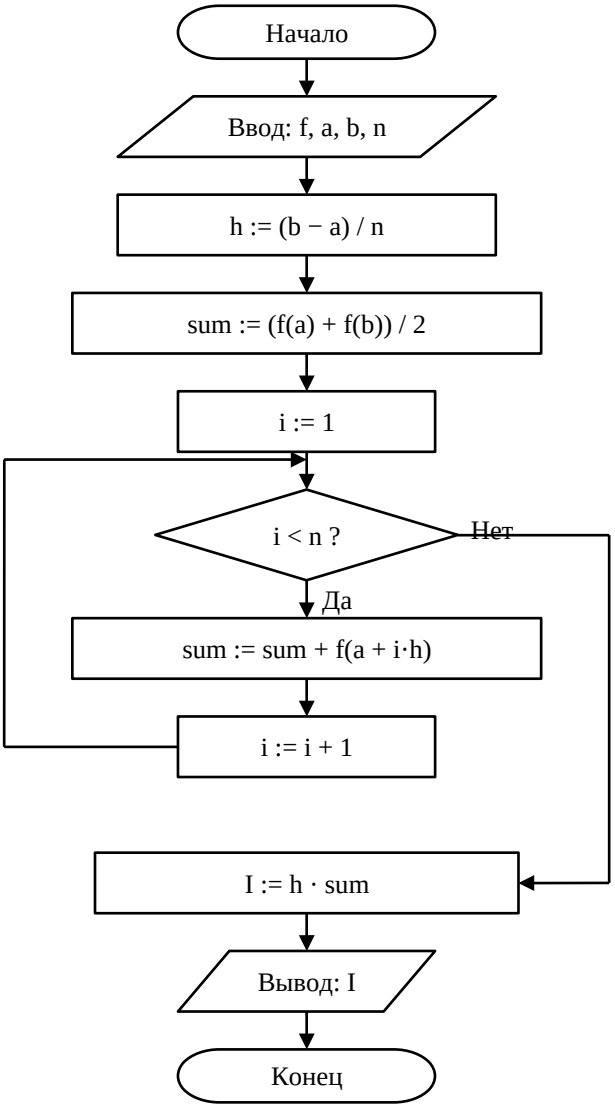


Рисунок 2.11 – Алгоритм метода Integrator.Trapezoid

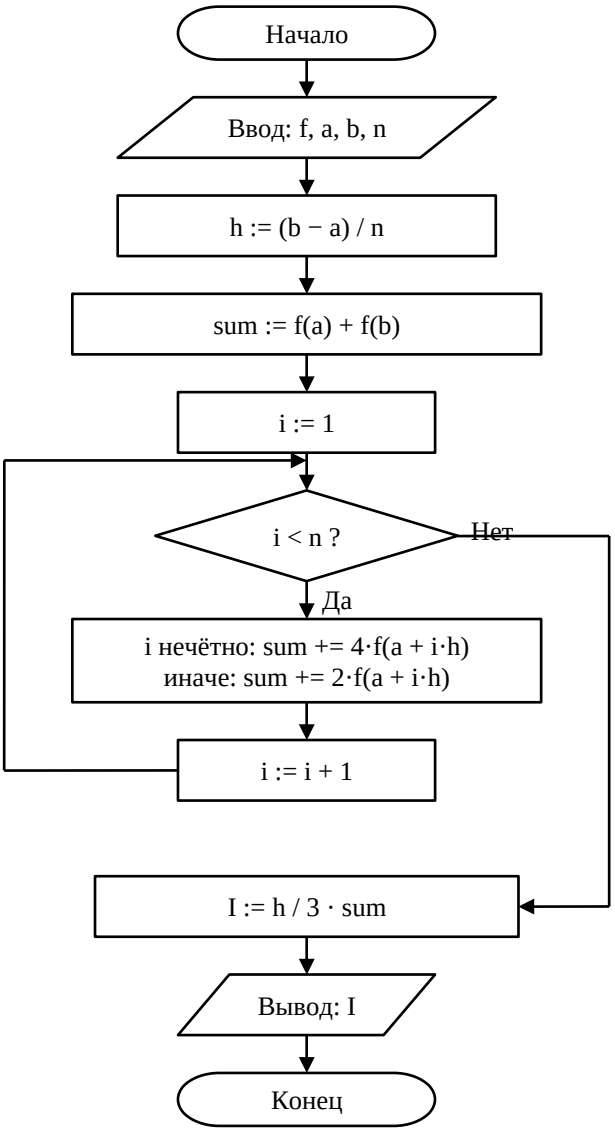


Рисунок 2.12 – Алгоритм метода Integrator.Simpson

3 Реализация и тестирование программы

3.1 Описание подпрограмм

Подпрограммы программы – методы классов. Вычислительные методы ядра не используют элементы управления и возвращают результат или бросают исключение с кодом ошибки; методы форм только читают поля, вызывают ядро и показывают результат. Основные подпрограммы приведены в таблице 3.1; при записи указан класс, затем метод. Подпрограммы, не требующие параметров, работают с полями своего класса.

Таблица 3.1 – Основные подпрограммы

Подпрограмма	Параметры	Результат	Назначение
`Lexer.Next`	—	Token	Выделяет следующую лексему формулы
`Lexer.ReadNumber`	—	Token	Читает число (целое, дробное, с порядком)
`Lexer.ReadWord`	—	Token	Читает слово: x, константа или имя функции
`Parser.Parse`	string formula	Node	Разбирает формулу в дерево выражения
`Parser.UsesX`	Node node	bool	Проверяет, входит ли x в выражение
`Parser.ParseExpression`	—	Node	Сумма и разность слагаемых
`Parser.ParseTerm`	—	Node	Произведение и частное множителей
`Parser.ParseFactor`	—	Node	Множитель со знаком
`Parser.ParsePower`	—	Node	Возведение в степень (правая ассоциативность)
`Parser.ParseBase`	—	Node	Число, x, константа, функция или скобки
`Parser.Expect`	TokenType type	—	Проверяет ожидаемую лексему и переходит к следующей
`Evaluator.Evaluate`	Node node, double x	double	Вычисляет дерево выражения в точке x
`Evaluator.Binary`	BinaryNode b, double x	double	Вычисляет бинарную операцию с проверками
`Evaluator.Function`	FunctionNode f, double x	double	Вычисляет функцию с проверкой области определения
`Evaluator.Finite`	double v, double x	double	Проверяет, что результат — конечное число
`Functions.IsFunction`	string name	bool	Проверяет, что имя — встроенная функция
`Functions.TryGetConstant`	string name, out double value	bool	Возвращает значение константы по имени
`Functions.TryApply`	string name, double arg, out double result	ErrorCode	Вычисляет функцию, проверяя область определения
`Integrator.Integrate`	Node f, IntegrationParams p, Method method	IntegrationResult	Основной алгоритм: интегрирование с подбором n по точности
`Integrator.Trapezoid`	Func f, double a, double b, int n	double	Составная формула трапеций
`Integrator.Simpson`	Func f, double a, double b, int n	double	Составная формула Симпсона (n чётное)
`Integrator.Validate`	IntegrationParams p	—	Проверка пределов, n и точности

Подпрограмма	Параметры	Результат	Назначение
`Integrator.ObserveOrder`	IntegrationResult res, IList steps, int order	—	Определяет фактический порядок сходимости
`TaskFile.Parse`	string text	IntegrationTask	Разбирает файл исходных данных
`TaskFile.ToText`	IntegrationTask t	string	Формирует текст файла исходных данных
`TaskFile.Assign`	IntegrationTask task, string key, string value, int line	—	Присваивает значение ключа с проверкой
`TaskFile.ReadNumber`	string value, string key, int line	double	Читает число из значения ключа
`TaskFile.ReadMethod`	string value, string key, int line	MethodChoice	Читает название метода
`NumberParser.TryParse`	string text, out double value	bool	Читает число из поля ввода (точка или запятая)
`AxisScale.NiceStep`	double min, double max, int target	double	Подбирает шаг делений оси из ряда 1, 2, 5 x 10^k
`AxisScale.Ticks`	double min, double max, int target	IList<double>	Значения делений оси внутри диапазона
`CalcForm.TryReadTask`	out IntegrationTask task, out Node node	bool	Читает и проверяет поля окна «Расчёт»
`CalcForm.Calculate`	—	—	Выполняет расчёт выбранным методом (или обоими) и выводит результаты
`CalcForm.LoadTask`	IntegrationTask t	—	Заполняет поля ввода из файла исходных данных
`CalcForm.ShowResults`	—	—	Выводит сводную таблицу и предупреждения
`CalcForm.ShowSteps`	—	—	Выводит таблицу сходимости выбранного метода
`PlotPanel.OnPaint`	PaintEventArgs e	—	Рисует график: сетку, площадь, узлы, кривую
`PlotPanel.DrawGrid`	Graphics g, Rectangle plot, ...	—	Рисует сетку и подписи осей
`MainForm.NewCalc`	IntegrationTask task, string title	CalcForm	Открывает новое окно расчёта
`MainForm.OpenData`	—	—	Загружает файл исходных данных в новое окно
`MainForm.SaveData`	—	—	Сохраняет исходные данные в файл *.txt
`MainForm.ShowGraph`	—	—	Открывает окно с графиком
`MainForm.ShowHelp`	int section	—	Открывает справку на нужном разделе
`MainForm.UpdateCommands`	—	—	Включает и выключает команды меню и панели инструментов
`IconFactory.Create`	Kind kind, int size	Bitmap	Рисует значок панели инструментов
`Messages.Text`	ErrorCode code, params object[] args	string	Возвращает готовую русскую фразу об ошибке
`Messages.Num`	double value	string	Форматирует число для сообщений

3.2 Управляющая программа и обработчики событий

Точкой входа служит метод Program.Main. Он включает визуальные стили, устанавливает обработчики необработанных исключений и запускает цикл сообщений Application.Run с главным окном MainForm. Если в обработчике события произойдёт непредвиденная ошибка, пользователь увидит сообщение, а программа продолжит работу, а не закроется аварийно.

Главное окно связывает каждую команду с одним действием (метод Wire): пункт меню и кнопка панели инструментов вызывают один и тот же обработчик, поэтому они всегда делают одно и то же. Доступность команд определяется методом UpdateCommands при смене активного окна: если нет открытых расчётов, команды сохранения, экспорта, расчёта и графика отключены и в меню, и на панели. Основные события и их обработчики приведены в таблице 3.2.

Таблица 3.2 – Обработчики основных событий

Событие	Обработчик	Действие
`Ctrl+N, «Новый расчёт»`	MainForm.NewCalc	открывает новое дочернее окно расчёта с примером x^2 на [0; 1]
`Ctrl+O, «Открыть данные»`	MainForm.OpenData	диалог выбора файла; TaskFile.Parse; данные попадают в новое окно; ошибка файла – сообщение с номером строки
`Ctrl+S, «Сохранить данные»`	MainForm.SaveData	записывает текущие поля в файл *.txt (TaskFile.ToText)
`Ctrl+E, «Экспорт»`	MainForm.ExportHtml	при устаревших результатах предлагает пересчитать; сохраняет отчёт и открывает его в браузере
`F5, кнопка «Вычислить»`	CalcForm.Calculate	читает поля (TryReadTask), вызывает Integrator.Integrate для выбранных методов, выводит результаты
`F6, «Показать график»`	MainForm.ShowGraph	при необходимости выполняет расчёт; открывает окно графика по модели PlotModel
`F1, «Содержание»`	MainForm.ShowHelp	открывает справку; из окна расчёта – раздел, соответствующий активному полю
`Изменение любого поля`	CalcForm.MarkStale	помечает результаты как устаревшие и выводит предупреждение
`Перетаскивание файла в окно`	MainForm.OnFileDragDrop	открывает перетащенные файлы *.txt, как команда «Открыть данные»
`Смена активного окна`	MainForm.UpdateCommands	включает и отключает команды, показывает метод в строке состояния

Порядок работы обработчика расчёта (CalcForm.Calculate) соответствует общей схеме алгоритма на рисунке 2.5. Сначала метод TryReadTask разбирает формулу методом Parser.Parse и по очереди проверяет поля a, b, n и ε. При первой найденной ошибке он показывает диалог с понятным текстом, переводит фокус в неверное поле, выделяет его содержимое (для формулы – ставит курсор на позицию неверного символа) и выводит сообщение в строке состояния. Если поля верны, для выбранного метода (или для обоих) вызывается Integrator.Integrate; исключения ядра (например, деление на ноль в некоторой точке отрезка) перехватываются, результаты очищаются, пользователь получает сообщение с указанием значения x.

При успешном расчёте результаты выводятся в таблицы, а данные запоминаются, чтобы позднее определить, не были ли они изменены.

3.3 Тестирование программы

3.3.1 Автоматические модульные тесты

Вычислительное ядро проверяется модульными тестами xUnit, которые запускаются командой dotnet test без запуска интерфейса. Наборы тестов приведены в таблице 3.3; всего выполняется 146 проверок, все они успешны. Контрольные интегралы с известным значением (многочлены, sin x, ln x, $4/(1+x^2)$, $\exp(-x^2)$) проверяют точность методов, а отношение погрешностей при удвоении n проверяет порядки 2 и 4: оно равно примерно 4 для формулы трапеций и 16 для формулы Симпсона, что подтверждает теорию из раздела 1. Отдельный тест сравнивает оценку погрешности по правилу Рунге с истинной ошибкой и допускает расхождение не более 25 %.

Таблица 3.3 – Наборы модульных тестов

Набор	Что проверяется
LexerTests	числа, имена, регистр, знаки; ошибки «недопустимый символ» и «неверное число» с позицией
ParserTests	приоритеты и ассоциативность (2+3*4, 2^3^2, −2^2); ошибки скобок, имён, лишних лексем; пустая формула
EvaluatorTests	значения функций и констант; все ошибки области определения со значением x
IntegratorTests	контрольные интегралы обоими методами; порядки сходимости; правило Рунге; a > b, a = b; нечётное n; предел n
TaskFileTests	корректный файл, значения по умолчанию, каждая ошибка с номером строки, BOM, CRLF, обратимость ToText и Parse
ReportTests	модель графика (диапазоны, разрывы, полюс, узлы), корректность SVG, формат чисел, разделы HTML-отчёта, экранирование
UiHelpersTests	разбор чисел из полей ввода, подбор делений оси графика
CasesTests	тестовый пример: 23 файла из каталога data/tests (см. 3.3.2)

3.3.2 Тестовый пример

Тестовые данные подготовлены в виде набора файлов в каталоге data/tests: 11 корректных задач и 12 ошибочных. Эталонные значения интегралов получены независимо от программы – вычислением с точностью 30 десятичных знаков средствами библиотеки mpmath языка Python, – а не взяты из вывода самой программы. Для каждого файла в манифесте указаны ожидаемое значение и допуск или ожидаемый код ошибки; прогон выполняется командой integ_cli --cases data/tests. Результаты прогона приведены в таблице 3.4.

Таблица 3.4 – Протокол выполнения тестового примера

№	Файл	Что проверяется	Ожидается	Получено	Итог
1	ok01_x2.txt	многочлен, оба метода	0.3333333333 (допуск 1e-7)	трапеции 0.3333333433; Симпсон 0.3333333333	верно
2	ok02_cubic.txt	многочлен 3-й степени	3.75 (допуск 1e-7)	трапеции 3.750000006; Симпсон 3.75	верно

№	Файл	Что проверяется	Ожидается	Получено	Итог
3	ok03_sin.txt	тригонометрия, число пи в пределах	2 (допуск 1e-7)	трапеции 1.999999994; Симпсон 2.000000004	верно
4	ok04_ln.txt	логарифм и константа e	1 (допуск 1e-7)	трапеции 0.9999999907; Симпсон 0.9999999945	верно
5	ok05_pi.txt	дробь, результат равен пи	3.141592654 (допуск 1e-7)	трапеции 3.141592644; Симпсон 3.141592651	верно
6	ok06_gauss.txt	интеграл Гаусса	0.8820813908 (допуск 1e-7)	трапеции 0.8820813849; Симпсон 0.8820813869	верно
7	ok07_task.txt	функция из задания, негладкая у нуля	41.80692254 (допуск 1e-4)	трапеции 41.80692104; Симпсон 41.80691784	верно
8	ok08_reversed.txt	a больше b	-0.5 (допуск 1e-12)	трапеции -0.5; Симпсон -0.5	верно
9	ok09_zero_width.txt	a равно b	0 (допуск 1e-12)	трапеции 0; Симпсон 0	верно
10	ok10_simpson_odd.txt	нечётное n у Симпсона	0.3333333333 (допуск 1e-12)	Симпсон 0.3333333333	верно
11	ok11_fixed_n.txt	фиксированное n без точности	2 (допуск 1e-5)	трапеции 1.999998355	верно
12	err01_paren.txt	пропущена закрывающая скобка (внутри формулы)	ошибка FileBadFormula	ошибка FileBadFormula: Строка 1: ошибка в формуле. Ожидается ")" в позиции 6.	верно
13	err02_unknown_func.txt	неизвестная функция (внутри формулы)	ошибка FileBadFormula	ошибка FileBadFormula: Строка 1: ошибка в формуле. Неизвестное имя "sn" в позиции 1.	верно
14	err03_no_operator.txt	пропущен знак операции (внутри формулы)	ошибка FileBadFormula	ошибка FileBadFormula: Строка 1: ошибка в формуле. Недопустимая лексема в позиции 2: ожидается знак операции или конец выражения.	верно
15	err04_bad_char.txt	недопустимый символ (внутри формулы)	ошибка FileBadFormula	ошибка FileBadFormula: Строка 1: ошибка в формуле. Недопустимый символ "@" в позиции 2.	верно
16	err05_div_zero.txt	деление на ноль в узле	ошибка DivByZero	ошибка DivByZero: Деление на ноль при x = 0.	верно
17	err06_ln_domain.txt	логарифм неположительного числа	ошибка LnDomain	ошибка LnDomain: Логарифм определён только для аргумента > 0 (x = -1).	верно
18	err07_empty.txt	пустой файл	ошибка FileEmpty	ошибка FileEmpty: Файл не содержит данных.	верно
19	err08_missing_b.txt	нет обязательного ключа b	ошибка FileMissingKey	ошибка FileMissingKey: Не задан обязательный ключ "b".	верно
20	err09_unknown_key.txt	неизвестный ключ	ошибка FileUnknownKey	ошибка FileUnknownKey: Строка 4: неизвестный ключ "step".	верно
21	err10_zero_n.txt	n равно нулю	ошибка FileBadValue	ошибка FileBadValue: Строка 4: неверное значение ключа "n".	верно

№	Файл	Что проверяется	Ожидается	Получено	Итог
22	err11_duplicate.txt	ключ указан дважды	ошибка FileDuplicateKey	ошибка FileDuplicateKey: Строка 2: ключ "function" указан повторно.	верно
23	err12_bad_method.txt	неизвестный метод	ошибка FileBadValue	ошибка FileBadValue: Строка 4: неверное значение ключа "method".	верно

Функция из задания $20 \cdot \sin(3\sqrt{x})$ на отрезке [0; 10] негладкая в нуле, поэтому при заданной точности 10^{-6} метод Симпсона даёт ошибку около $4{,}7 \cdot 10^{-6}$, а оценка по правилу Рунге – $5{,}7 \cdot 10^{-7}$: оценка занижена примерно в 8 раз, у формулы трапеций – в 1,6 раза. Программа обнаруживает снижение фактического порядка сходимости и предупреждает пользователя, поэтому допуск для этого файла в манифесте равен 10^{-4}, а не 10^{-7}.

3.3.3 Проверка работы интерфейса

Реакция программы на ошибки пользователя показана на рисунках 3.1 и 3.2: это сообщения об ошибке в формуле и о делении на ноль в узле разбиения; на рисунке 3.3 – отчёт в браузере для функции из задания.

[Здесь будет снимок: сообщение об ошибке в формуле (файл err01_paren.txt) – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 3.1 – Сообщение об ошибке в формуле

[Здесь будет снимок: сообщение о делении на ноль (файл err05_div_zero.txt) – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 3.2 – Сообщение о нарушении области определения

[Здесь будет снимок: HTML-отчёт для функции из задания в браузере – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок 3.3 – Отчёт в формате HTML

ВЫВОДЫ

В ходе курсовой работы разработана программа «Интегрирование функций» для вычисления определённых интегралов функций, заданных формулой, методами трапеций и Симпсона. Программа соответствует заданию: ввод исходных данных из текстового файла, выбор метода, вывод результатов на форму и в файл формата HTML, многооконный интерфейс с главным меню, панелью инструментов, строкой состояния и справкой.

Для разбора формулы реализован собственный лексический и синтаксический анализатор методом рекурсивного спуска; значение выражения вычисляется обходом дерева с проверкой области определения, а ошибки ввода сопровождаются позицией в формуле или номером строки файла. Точность расчёта может задаваться числом разбиений либо требуемой погрешностью с автоматическим подбором числа разбиений по правилу Рунге.

Тестирование показало, что при гладких функциях порядки сходимости соответствуют теории: отношение погрешностей при удвоении числа разбиений близко к 4 для формулы трапеций и к 16 для формулы Симпсона. Для негладкой функции из задания эти порядки не достигаются, оценка погрешности занижается, и программа предупреждает об этом пользователя. В ходе проверки найдена и исправлена ошибка: при расчёте с заданным числом разбиений оценка погрешности была занижена в 4 раза для трапеций и в 16 раз для Симпсона.

Вычислительное ядро отделено от интерфейса и проверено автоматическими модульными тестами (146 проверок) и тестовым примером из 23 файлов, эталоны которого получены независимо от программы. Программа может быть расширена другими квадратурными формулами или другими видами подынтегральных функций без изменения интерфейса.

СПИСОК ИСТОЧНИКОВ

1. Бахвалов Н.С., Жидков Н.П., Кобельков Г.М. Численные методы. – 3-е изд. – М.: БИНОМ, 2004. – 636 с.

2. Демидович Б.П., Марон И.А. Основы вычислительной математики. – 6-е изд. – СПб.: Лань, 2007. – 672 с.

3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005. – 272 с.

ПРИЛОЖЕНИЕ А. ЭКРАННЫЕ ФОРМЫ

Экранные формы программы, не вошедшие в основной текст.

[Здесь будет снимок: окно справки (F1) – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок А.1 – Окно справки

[Здесь будет снимок: окно «О программе» – снимается на Windows, см. docs/phases/F6_tests.md]

Рисунок А.2 – Окно «О программе»

ПРИЛОЖЕНИЕ Б. ЛИСТИНГ ПРОГРАММЫ

Приведены основные модули вычислительного ядра. Полный текст проекта находится на диске, прилагаемом к работе.

Файл src/Integral.Core/Integrator.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;

namespace Integral.Core
{
    /// <summary>Численное интегрирование: формулы трапеций и Симпсона, оценка точности по
Рунге.</summary>
    public static class Integrator
    {
        /// <summary>Вычисляет интеграл функции f на [A; B]. Бросает <see
cref="IntegralException"/>.</summary>
        public static IntegrationResult Integrate(Node f, IntegrationParams p, Method method)
        {
            Validate(p);
            var sw = Stopwatch.StartNew();
            long evals = 0;
            Func<double, double> fn = x => { evals++; return Evaluator.Evaluate(f, x); };

            int order = method == Method.Simpson ? 4 : 2;
            double divisor = Math.Pow(2, order) - 1;
            var res = new IntegrationResult { Method = method };

            int n = Math.Max(p.N, method == Method.Simpson ? 2 : 1);
            if (method == Method.Simpson && n % 2 == 1) { n++; res.NAdjusted = true; }
            if (p.Eps > 0 && n < 2) n = 2;

            if (p.A == p.B)
            {
                res.Value = 0; res.N = n; res.H = 0; res.ErrorEstimate = 0;
                res.ElapsedMs = sw.Elapsed.TotalMilliseconds;
                return res;
            }

            Func<int, double> compute = k => method == Method.Simpson ? Simpson(fn, p.A, p.B, k) :
Trapezoid(fn, p.A, p.B, k);
            var steps = new List<ConvergenceStep>();
            double prev = compute(n);
            steps.Add(new ConvergenceStep(n, prev, double.NaN));
            res.Value = prev; res.N = n;

            if (p.Eps <= 0)
            {
                // Фиксированное n: оценка по Рунге, если удвоение не выходит за предел. Формула
                // |Q(2n) − Q(n)| / (2^p − 1) оценивает ошибку уточнённого значения Q(2n); ошибка
выводимого
                // значения Q(n) в 2^p раз больше, поэтому умножаем (иначе оценка занижена в 4 или 16
раз).
                if (n * 2 <= IntegrationParams.MaxN)
                    res.ErrorEstimate = (divisor + 1) * Math.Abs(compute(n * 2) - prev) / divisor;
            }
            else
            {
                res.Converged = false;
                while (n * 2 <= IntegrationParams.MaxN)
                {
                    int n2 = n * 2;
```

```
                    double cur = compute(n2);
                    double r = Math.Abs(cur - prev) / divisor;
                    steps.Add(new ConvergenceStep(n2, cur, r));
                    res.Value = cur; res.N = n2; res.ErrorEstimate = r;
                    if (r <= p.Eps) { res.Converged = true; break; }
                    prev = cur; n = n2;
                }
            }

            res.H = (p.B - p.A) / res.N;
            res.Steps = steps;
            ObserveOrder(res, steps, order);
            res.Evaluations = evals;
            res.ElapsedMs = sw.Elapsed.TotalMilliseconds;
            return res;
        }

        /// <summary>Фактический порядок: log2((I(n)−I(n/2)) / (I(2n)−I(n))) по трём последним
шагам.</summary>
        private static void ObserveOrder(IntegrationResult res, IList<ConvergenceStep> s, int order)
        {
            int k = s.Count;
            if (k < 3) return;
            double d1 = s[k - 2].Value - s[k - 3].Value;
            double d2 = s[k - 1].Value - s[k - 2].Value;
            if (d2 == 0 || d1 / d2 <= 0) return;
            res.ObservedOrder = Math.Log(d1 / d2, 2);
            res.OrderDegraded = res.ObservedOrder < 0.8 * order;
        }

        /// <summary>Составная формула трапеций: h·[(f0+fn)/2 + Σ fi].</summary>
        public static double Trapezoid(Func<double, double> f, double a, double b, int n)
        {
            double h = (b - a) / n;
            double sum = (f(a) + f(b)) / 2;
            for (int i = 1; i < n; i++) sum += f(a + i * h);
            return h * sum;
        }

        /// <summary>Составная формула Симпсона (n чётное): h/3·[f0 + 4Σf_нечёт + 2Σf_чёт +
fn].</summary>
        public static double Simpson(Func<double, double> f, double a, double b, int n)
        {
            double h = (b - a) / n;
            double sum = f(a) + f(b);
            for (int i = 1; i < n; i++) sum += (i % 2 == 1 ? 4 : 2) * f(a + i * h);
            return h / 3 * sum;
        }

        private static void Validate(IntegrationParams p)
        {
            if (double.IsNaN(p.A) || double.IsInfinity(p.A) || double.IsNaN(p.B) ||
double.IsInfinity(p.B))
                throw new ParamsException(ErrorCode.BadLimits);
            if (p.N < 1 || p.N > IntegrationParams.MaxN)
                throw new ParamsException(ErrorCode.BadN, IntegrationParams.MaxN);
            if (double.IsNaN(p.Eps) || double.IsInfinity(p.Eps) || p.Eps < 0)
                throw new ParamsException(ErrorCode.BadEps);
        }
    }
}
```

Файл src/Integral.Core/Evaluator.cs

```
using System;
```

```
namespace Integral.Core
{
    /// <summary>Вычисление дерева выражения в точке x с контролем области определения.</summary>
    public static class Evaluator
    {
        /// <summary>Значение выражения при заданном x. Бросает <see cref="EvalException"/>.</summary>
        public static double Evaluate(Node node, double x)
        {
            switch (node)
            {
                case NumberNode n: return n.Value;
                case VariableNode _: return x;
                case NegateNode g: return -Evaluate(g.Operand, x);
                case BinaryNode b: return Binary(b, x);
                case FunctionNode f: return Function(f, x);
                default: throw new ArgumentException("Неизвестный тип узла.", nameof(node));
            }
        }

        private static double Binary(BinaryNode b, double x)
        {
            double l = Evaluate(b.Left, x);
            double r = Evaluate(b.Right, x);
            double res;
            switch (b.Op)
            {
                case '+': res = l + r; break;
                case '-': res = l - r; break;
                case '*': res = l * r; break;
                case '/':
                    if (r == 0) throw new EvalException(ErrorCode.DivByZero, x);
                    res = l / r;
                    break;
                case '^':
                    if (l == 0 && r < 0) throw new EvalException(ErrorCode.DivByZero, x);
                    if (l < 0 && r != Math.Floor(r)) throw new EvalException(ErrorCode.PowDomain, x);
                    res = Math.Pow(l, r);
                    break;
                default: throw new ArgumentException("Неизвестная операция.", nameof(b));
            }
            return Finite(res, x);
        }

        private static double Function(FunctionNode f, double x)
        {
            double arg = Evaluate(f.Argument, x);
            ErrorCode code = Functions.TryApply(f.Name, arg, out double res);
            if (code != ErrorCode.None) throw new EvalException(code, x);
            return Finite(res, x);
        }

        private static double Finite(double v, double x)
        {
            if (double.IsNaN(v) || double.IsInfinity(v)) throw new EvalException(ErrorCode.NotFinite,
x);
            return v;
        }
    }
}
```

Файл src/Integral.Core/Lexer.cs

```
using System;
using System.Globalization;
```

```
namespace Integral.Core
{
    /// <summary>Лексический анализатор формулы: выделяет лексемы по одной.</summary>
    public sealed class Lexer
    {
        private readonly string _src;
        private int _pos; // индекс следующего символа, с 0

        public Lexer(string source)
        {
            _src = source ?? string.Empty;
        }

        /// <summary>Возвращает следующую лексему; в конце строки — <see
cref="TokenType.End"/>.</summary>
        public Token Next()
        {
            while (_pos < _src.Length && char.IsWhiteSpace(_src[_pos])) _pos++;
            if (_pos >= _src.Length) return new Token(TokenType.End, _pos + 1, 0);

            char c = _src[_pos];
            int pos1 = _pos + 1;
            switch (c)
            {
                case '+': _pos++; return new Token(TokenType.Plus, pos1, 1);
                case '-': _pos++; return new Token(TokenType.Minus, pos1, 1);
                case '*': _pos++; return new Token(TokenType.Multiply, pos1, 1);
                case '/': _pos++; return new Token(TokenType.Divide, pos1, 1);
                case '^': _pos++; return new Token(TokenType.Power, pos1, 1);
                case '(': _pos++; return new Token(TokenType.LeftParen, pos1, 1);
                case ')': _pos++; return new Token(TokenType.RightParen, pos1, 1);
            }
            if (IsDigit(c)) return ReadNumber();
            if (c == '.' || c == ',') throw new FormulaException(ErrorCode.BadNumber, pos1, pos1);
            if (IsLetter(c)) return ReadWord();
            throw new FormulaException(ErrorCode.BadChar, pos1, c, pos1);
        }

        /// <summary>Число: цифры [.|, цифры] [e|E [+|-] цифры].</summary>
        private Token ReadNumber()
        {
            int start = _pos;
            while (_pos < _src.Length && IsDigit(_src[_pos])) _pos++;
            if (_pos < _src.Length && (_src[_pos] == '.' || _src[_pos] == ','))
            {
                _pos++;
                if (_pos >= _src.Length || !IsDigit(_src[_pos]))
                    throw new FormulaException(ErrorCode.BadNumber, start + 1, start + 1);
                while (_pos < _src.Length && IsDigit(_src[_pos])) _pos++;
            }
            // Экспонента принимается, только если за ней идут цифры; иначе "e" — отдельная константа.
            if (_pos < _src.Length && (_src[_pos] == 'e' || _src[_pos] == 'E'))
            {
                int p = _pos + 1;
                if (p < _src.Length && (_src[p] == '+' || _src[p] == '-')) p++;
                if (p < _src.Length && IsDigit(_src[p]))
                {
                    while (p < _src.Length && IsDigit(_src[p])) p++;
                    _pos = p;
                }
            }
            string text = _src.Substring(start, _pos - start).Replace(',', '.');
            double value = double.Parse(text, NumberStyles.Float, CultureInfo.InvariantCulture);
```

```
            if (double.IsInfinity(value)) throw new FormulaException(ErrorCode.BadNumber, start + 1,
start + 1);
            return new Token(TokenType.Number, start + 1, _pos - start, value);
        }

        /// <summary>Слово: x, константа, имя функции; регистр не учитывается.</summary>
        private Token ReadWord()
        {
            int start = _pos;
            while (_pos < _src.Length && (IsLetter(_src[_pos]) || IsDigit(_src[_pos]))) _pos++;
            string raw = _src.Substring(start, _pos - start);
            string word = raw.ToLowerInvariant();
            int len = _pos - start;
            if (word == "x") return new Token(TokenType.Variable, start + 1, len);
            if (Functions.TryGetConstant(word, out double c)) return new Token(TokenType.Constant,
start + 1, len, c, word);
            if (Functions.IsFunction(word)) return new Token(TokenType.Function, start + 1, len, 0,
word);
            throw new FormulaException(ErrorCode.UnknownIdent, start + 1, raw, start + 1);
        }

        private static bool IsDigit(char c) => c >= '0' && c <= '9';
        private static bool IsLetter(char c) => (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
    }
}
```

Файл src/Integral.Core/Parser.cs

```
namespace Integral.Core
{
    /// <summary>
    /// Синтаксический анализатор формулы (рекурсивный спуск по грамматике БНФ).
    /// Выражение  ::= Слагаемое { ('+'|'-') Слагаемое }
    /// Слагаемое  ::= Множитель { ('*'|'/') Множитель }
    /// Множитель  ::= ['+'|'-'] Степень
    /// Степень    ::= Основа [ '^' Множитель ]
    /// Основа     ::= Число | 'x' | Константа | Функция '(' Выражение ')' | '(' Выражение ')'
    /// </summary>
    public sealed class Parser
    {
        /// <summary>Максимальная вложенность разбора (защита от переполнения стека).</summary>
        public const int MaxDepth = 200;

        private readonly Lexer _lexer;
        private Token _cur;
        private int _depth;

        private Parser(string source)
        {
            _lexer = new Lexer(source);
            _cur = _lexer.Next();
        }

        /// <summary>Разбирает формулу в дерево выражения. Бросает <see
cref="FormulaException"/>.</summary>
        public static Node Parse(string formula)
        {
            var parser = new Parser(formula);
            if (parser._cur.Type == TokenType.End)
                throw new FormulaException(ErrorCode.EmptyExpression, 1);
            Node root = parser.ParseExpression();
            if (parser._cur.Type != TokenType.End)
                throw new FormulaException(ErrorCode.UnexpectedToken, parser._cur.Position,
parser._cur.Position);
            return root;
```

```
    }

    /// <summary>Есть ли в дереве переменная x.</summary>
    public static bool UsesX(Node node)
    {
        switch (node)
        {
            case VariableNode _: return true;
            case NegateNode n: return UsesX(n.Operand);
            case BinaryNode b: return UsesX(b.Left) || UsesX(b.Right);
            case FunctionNode f: return UsesX(f.Argument);
            default: return false;
        }
    }

    private void Advance() { _cur = _lexer.Next(); }

    private void Enter()
    {
        if (++_depth > MaxDepth)
            throw new FormulaException(ErrorCode.TooDeep, _cur.Position, MaxDepth);
    }

    private void Leave() { _depth--; }

    /// <summary>Выражение: сложение и вычитание слагаемых.</summary>
    private Node ParseExpression()
    {
        Enter();
        Node left = ParseTerm();
        while (_cur.Type == TokenType.Plus || _cur.Type == TokenType.Minus)
        {
            char op = _cur.Type == TokenType.Plus ? '+' : '-';
            Advance();
            left = new BinaryNode(op, left, ParseTerm());
        }
        Leave();
        return left;
    }

    /// <summary>Слагаемое: умножение и деление множителей.</summary>
    private Node ParseTerm()
    {
        Node left = ParseFactor();
        while (_cur.Type == TokenType.Multiply || _cur.Type == TokenType.Divide)
        {
            char op = _cur.Type == TokenType.Multiply ? '*' : '/';
            Advance();
            left = new BinaryNode(op, left, ParseFactor());
        }
        return left;
    }

    /// <summary>Множитель: необязательный унарный знак перед степенью.</summary>
    private Node ParseFactor()
    {
        Enter();
        Node result;
        if (_cur.Type == TokenType.Minus) { Advance(); result = new NegateNode(ParseFactor()); }
        else if (_cur.Type == TokenType.Plus) { Advance(); result = ParseFactor(); }
        else result = ParsePower();
        Leave();
        return result;
    }
```

```
        /// <summary>Степень: основа и необязательный показатель (правая ассоциативность).</summary>
        private Node ParsePower()
        {
            Node b = ParseBase();
            if (_cur.Type == TokenType.Power)
            {
                Advance();
                return new BinaryNode('^', b, ParseFactor());
            }
            return b;
        }

        /// <summary>Основа: число, x, константа, функция или выражение в скобках.</summary>
        private Node ParseBase()
        {
            Token t = _cur;
            switch (t.Type)
            {
                case TokenType.Number:
                case TokenType.Constant:
                    Advance();
                    return new NumberNode(t.Value);
                case TokenType.Variable:
                    Advance();
                    return new VariableNode();
                case TokenType.Function:
                    Advance();
                    if (_cur.Type != TokenType.LeftParen)
                        throw new FormulaException(ErrorCode.ExpectedLParen, _cur.Position, t.Name,
_cur.Position);
                    Advance();
                    Node arg = ParseExpression();
                    Expect(TokenType.RightParen);
                    return new FunctionNode(t.Name, arg);
                case TokenType.LeftParen:
                    Advance();
                    Node inner = ParseExpression();
                    Expect(TokenType.RightParen);
                    return inner;
                default:
                    throw new FormulaException(ErrorCode.ExpectedOperand, t.Position, t.Position);
            }
        }

        private void Expect(TokenType type)
        {
            if (_cur.Type != type)
                throw new FormulaException(ErrorCode.ExpectedRParen, _cur.Position, _cur.Position);
            Advance();
        }
    }
}
```

ПРИЛОЖЕНИЕ В. ФАЙЛЫ ТЕСТОВОГО ПРИМЕРА

Приведены примеры файлов исходных данных из каталога data/tests; остальные файлы находятся на диске.

Файл ok01_x2.txt

```
# Парабола: оба метода, Симпсон точен для многочленов до 3-й степени
function = x^2
a = 0
b = 1
method   = both
eps      = 1e-8
n        = 4
```

Файл ok07_task.txt

```
# Функция из примечания к заданию (негладкая у нуля: корень)
function = 20*sin(sqrt(x)*3)
a = 0
b = 10
method = both
n = 100
eps = 1e-6
```

Файл ok08_reversed.txt

```
# a > b: интеграл меняет знак
function = x
a = 1
b = 0
method = both
n = 10
```

Файл ok10_simpson_odd.txt

```
# Симпсон при нечётном n: программа берёт n+1
function = x^2
a = 0
b = 1
method = simpson
n = 5
```

Файл err01_paren.txt

```
function = sin(x
a = 0
b = 1
n = 10
```

Файл err05_div_zero.txt

```
function = 1/x
a = -1
b = 1
method = trapezoid
n = 10
```

Файл err08_missing_b.txt

```
function = x
a = 0
n = 10
```

Файл err10_zero_n.txt

```
function = x
```

```
a = 0
b = 1
n = 0
```